NBA Play Ranker Software Design Document

INFO45828 Computer Science Capstone Project
Alpha Release
Authors: Abdul Rahman Mousa (mousaa@sheridancollege.ca) and Gurkaranjit Asahan (asahang@sheridancollege.ca)
Supervisor: Prof. Cristina Ribeiro
Honours Bachelor of Computer Science, Faculty of Applied Science and Technology, Sheridan College
Fall 2026

Table of Contents

1. Introduction....6
2. Software Architecture....6
2.1 Overall Software Architecture....6
2.1.1 Deployment View....7
2.2 Software Architecture Class Diagram....10
2.2.1 Software System Architecture....10
2.2.2 Layer and Subsystem Dependencies....12
2.3 Layers and Subsystems....13
2.3.1 User Interface Layer....13
2.3.2 Application Layer....13
2.3.3 Domain Layer....13
2.3.4 Infrastructure Layer....13
3. Detailed Design....13
3.1 User Interface Layer....14
3.1.1 Data Explorer UI....14
3.1.2 Recommendation UI....15
3.1.3 Analytics UI....17
3.1.4 Authentication UI....18
3.1.5 Gameplan UI....19
3.1.6 Shot Analysis UI....20
3.2 Application Layer....21
3.2.1 API Coordination....21
3.2.2 Recommendation Services....24
3.2.3 Analytics Services....26
3.2.4 Access Control Services....27

3.2.5 Frontend Services....28
3.3 Domain Layer....28
3.3.1 Baseline Recommendation....29
3.3.2 Context ML Recommendation....30
3.3.3 Statistical Analysis....30
3.3.4 Shot Analysis....31
3.3.5 Frontend Domain....31
3.4 Infrastructure Layer....32
3.4.1 Data Access....32
3.4.2 Model Management....32
3.4.3 Visualization and Export....33
3.4.4 External Integrations....34
3.4.5 Frontend Infrastructure....35
4. Interaction Model....37
4.1 Compare Usage and Efficiency....38
4.1.1 Collaboration Overview - Compare Usage and Efficiency....38
4.1.2 Structural Design - Compare Usage and Efficiency....39
4.1.3 Detailed Sequence - Compare Usage and Efficiency....40
4.2 View Opponent Defensive Weak Spots....40
4.2.1 Collaboration Overview - View Opponent Defensive Weak Spots....41
4.2.2 Structural Design - View Opponent Defensive Weak Spots....42
4.2.3 Detailed Sequence - View Opponent Defensive Weak Spots....43
4.3 Access Analyst Workspace....44
4.3.1 Collaboration Overview - Access Analyst Workspace....44
4.3.2 Structural Overview - Access Analyst Workspace....44
4.3.3 Behavioural Overview - Access Analyst Workspace....45

4.4 Ranked List via API (Baseline)....48
4.4.1 Collaboration Overview - Ranked List via API (Baseline)....48
4.4.2 Structural Overview - Ranked List via API (Baseline)....49
4.4.3 Behavioural Overview - Ranked List via API (Baseline)....49
4.5 Rank Plays with Game Context....52
4.5.1 Collaboration Overview - Rank Plays with Game Context....52
4.5.2 Structural Overview - Rank Plays with Game Context....53
4.5.3 Behavioural Overview - Rank Plays with Game Context....53
4.6 Authenticate Analyst Access....56
4.6.1 Collaboration Overview - Authenticate Analyst Access....56
4.6.2 Structural Overview - Authenticate Analyst Access....56
4.6.3 Behavioural Overview - Authenticate Analyst Access....57
4.7 Validate Coach Access....60
4.7.1 Collaboration Overview - Validate Coach Access....60
4.7.2 Structural Overview - Validate Coach Access....60
4.7.3 Behavioural Overview - Validate Coach Access....61
4.8 Export KPIs / Reports....63
4.8.1 Collaboration Overview - Export KPIs / Reports (486)....64
4.9 Explore and Export Play-by-Play Shot Data....64
4.9.1 Collaboration Overview - Explore and Export Play-by-Play Shot Data....65
4.9.2 Structural Overview - Explore and Export Play-by-Play Shot Data....65
4.9.3 Behavioural Overview - Explore and Export Play-by-Play Shot Data....66
4.10 Highlight Top Pick....67
4.10.1 Collaboration Overview - Highlight Top Pick....68
4.10.2 Structural Overview - Highlight Top Pick....68
4.10.3 Behavioural Overview - Highlight Top Pick....69

4.11 Search Endpoint Activity Log....70
4.11.1 Collaboration Overview - Search Endpoint Activity Log....71
4.11.2 Structural Overview - Search Endpoint Activity Log....71
4.11.3 Behavioural Overview - Record and Search Request Log....72
4.12 Timeout Quick-Call Mode....74
4.12.1 Collaboration Overview - Timeout Quick-Call Mode....75
4.12.2 Structural Overview - Timeout Quick-Call Mode....75
4.13 Sync Gameplan Notes Across Devices....76
4.13.1 Collaboration Overview - Sync Gameplan Notes Across Devices....77

1. Introduction

This document describes the software design of NBA Play Ranker, a decision-support web application that recommends offensive play types for a matchup and explains why. It is written for developers and reviewers who need to understand how the system is built, not only what it does. The Software Requirements Specification (SRS) covers the requirements, so this document focuses on the software architecture, the detailed design of each layer and subsystem, and the interactions that realize the use cases.

The document is organised in three parts. The Software Architecture section explains the overall structure, why it was chosen and how it is deployed, and lists the layers and subsystems. The Detailed Design section goes through each layer and subsystem with its class diagram, and the detailed sequence diagrams that belong to it. The Interaction Model section then lists every use case interaction with its collaboration overview, structural (class) design and behavioural (sequence) design. Release scope. Beta: SCRUM-697 adds dedicated league comparisons; SCRUM-698 adds regular-season trends. These are planned, not part of the Alpha completion claim. Final: SCRUM-699 adds side-by-side team profiles; SCRUM-700 adds independent possession outcome rates. These are planned, not part of the Alpha completion claim. Alpha testing checks source totals, season isolation, aggregation denominators, ranking direction, missing categories, sample cutoffs, access control, filter changes, loading and retry behavior. Initial failures remain linked to their repair tickets. Automated checks and browser checks establish the tested behavior; they do not replace the team's recorded demonstration or a fresh domain-expert review.

2. Software Architecture

2.1 Overall Software Architecture

We chose a layered architecture because the work NBA Play Ranker does falls naturally into four separate concerns that change at different rates. The hoopR data pulls, the parquet and CSV files and the Supabase services change the most, so they are kept in the Infrastructure layer where a change in a source schema or a provider does not ripple up into the ranking logic. The basketball rules (possession-weighted PPP, shrinkage toward the league value, the context overlays and the top pick rule) sit in the Domain layer and do not know where the data came from. The Application layer coordinates each request, checks the role and turns domain results into API responses, and the User Interface layer only renders what it is given.

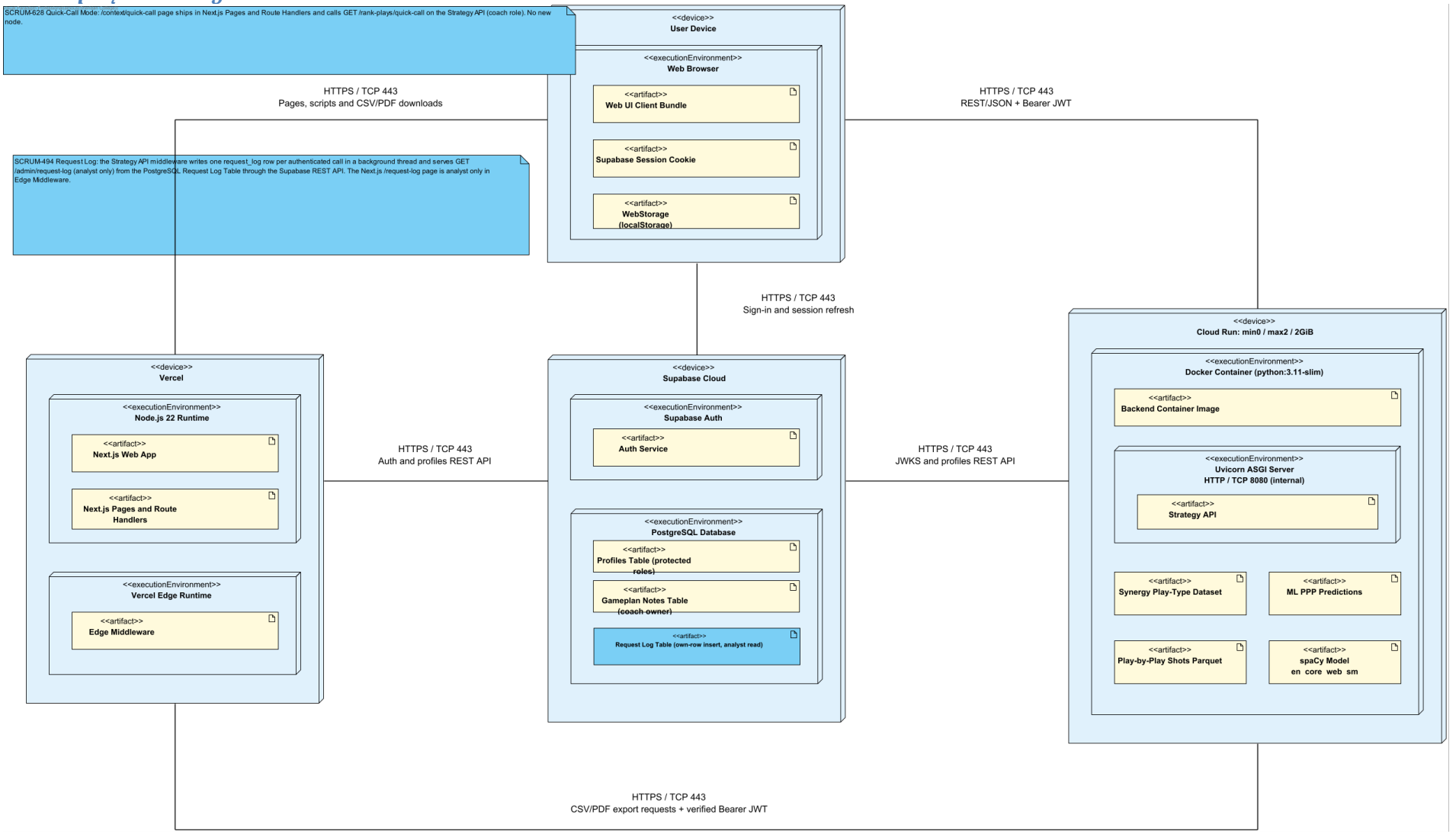
A microservices design was considered but rejected. The system is built and run by a two-person team, all of the services read the same cached datasets, and splitting them across services would add network calls, deployment work and consistency problems without giving us any real scaling benefit. A pipe-and-filter design fits the offline data pipeline (pull, clean, aggregate, train), and we do use that style inside the build scripts, but it does not fit the interactive part of the product, where a coach changes a filter and expects a ranking, an explanation and an export that all agree with each other. The layered design gives us that consistency, and it supports the non-functional requirements directly: each layer can be tested on its own, the expensive data work is cached below the API so responses stay fast, and a failure in one integration (for example Supabase being paused) is handled in its adapter instead of breaking the whole page.

Several design patterns are used inside the layers. Data access classes such as the PBP shot store and the Synergy loaders follow the Repository pattern, so the rest of the system asks for rows by season and team without knowing about files or caching. The Supabase stores (request log, gameplan sync) act as Adapters around the external service. The API routers act as a Facade over the services in the Application layer, and the export contract uses a single builder so that CSV, JSON and PDF are always produced from the same rows.

2.1.1 Deployment View

The browser reaches the Next.js frontend on Vercel and the FastAPI service on Google Cloud Run over HTTPS on port 443. Supabase provides authentication and PostgreSQL-backed application storage. The Cloud Run container listens on its configured internal port. Offline scripts prepare model and analysis artifacts; normal dashboard requests do not train models. The existing deployment diagram records the service boundaries and labelled protocols.

2.1.1.1 Deployment Diagram

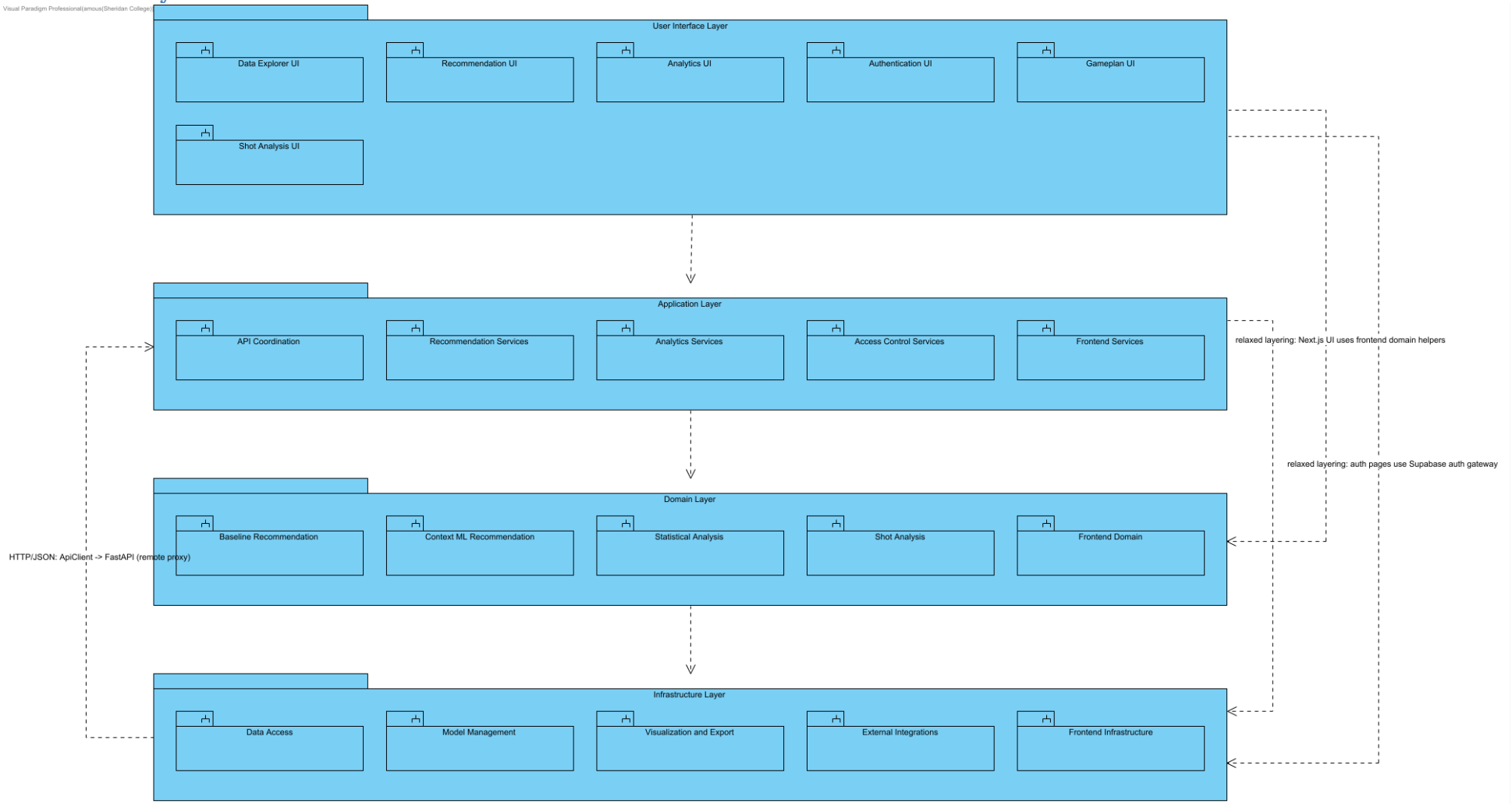


SCRUM-506. Frontend https://nbaplayranker-seven.vercel.app on Vercel Hobby, Node.js 22, manually deployed from canonical shared Git checkout. Cloud Run nba-playranker-api in northamerica-northeast1: Python3.11/Uvicorn application.api_coordination.app:app, 1CPU/2GiB, min0/max2, concurrency2, numeric threads1. HTTPS REST/JSON + Bearer and explicit frontend CORS. Supabase Free Auth/JWKS, protected profiles and coach-owned gameplan_notes RLS. Analyst uplift/calibration reports use packaged datasets and ephemeral instance cache. Coach Gameplan notes, plans and roles synchronize with the coach-owned

Supabase table through authenticated owner-only REST requests. CAD10 budget alerts and CAD7 Cloud Run cap have delayed enforcement, not an absolute total billing guarantee.Production authorization validates ES256/RS256 JWT signatures against Supabase JWKS plus issuer, audience and expiry; verified subject identifies the user. The current protected public.profiles.role authorizes resources, never user_metadata.role. Unassigned accounts remain pending. Analyst evidence endpoints require analyst access; coach recommendations and the matchup dashboard require coach access. Shared shot views accept either approved role. Invalid tokens return 401 and forbidden roles return 403.Deployment view: artifacts are grouped inside their execution environments. Six host communication paths show HTTPS on TCP port 443. Cloud Run terminates TLS and forwards HTTP to Uvicorn on internal port 8080. Supabase access uses HTTPS Auth and REST APIs; no direct PostgreSQL connection is used by the app. Browser Gameplan state synchronizes with Supabase using INSERT for a new matchup and revision-filtered updates; local draft recovery remains available for conflicts or interrupted edits. Detailed deploy, composition and artifact dependency relationships remain in the model and are hidden only on this view to avoid duplicate arrows.

2.2 Software Architecture Class Diagram

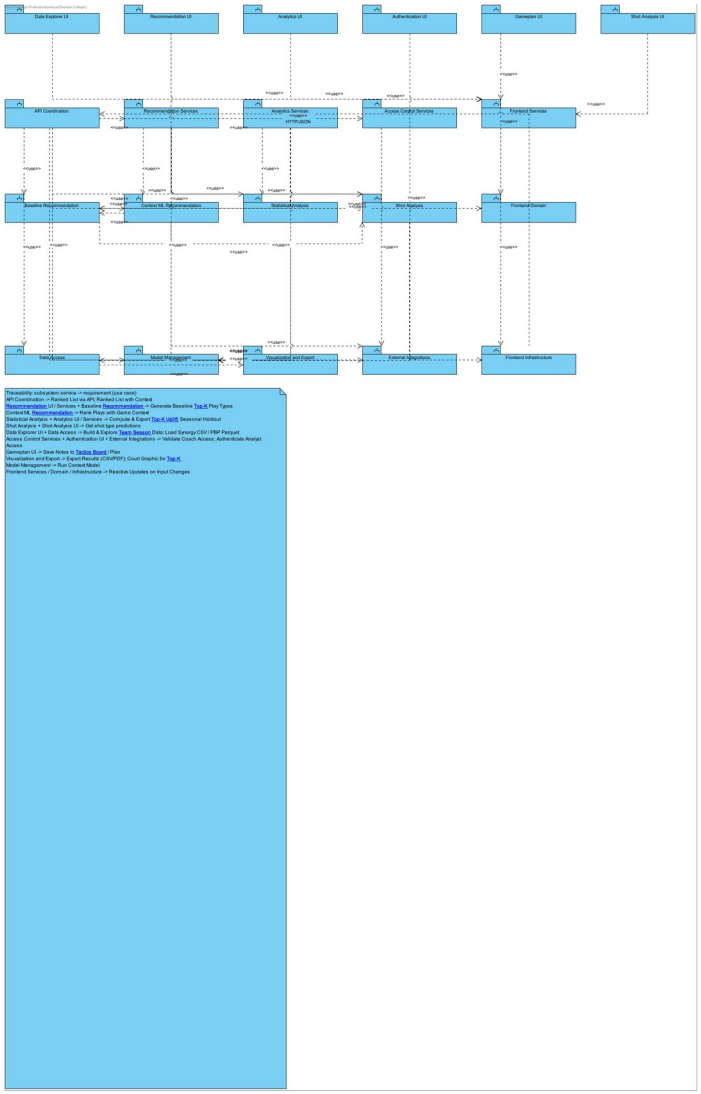
2.2.1 Software System Architecture



System type: NBA Play Ranker is a web-based decision-support information system. Coaches and analysts select a matchup (season, our team, opponent) and optionally a game situation, and the system returns a ranked Top-K list of offensive play types with explanations, visual evidence (court zone and shot heatmaps) and CSV/PDF exports.Architecture: a four-layer architecture (User Interface -> Application -> Domain -> Infrastructure) implemented across two deployable tiers: a Next.js/React frontend (Vercel) and a Python FastAPI backend, with Supabase providing authentication and user roles. Each layer is decomposed into subsystems;

each subsystem exposes public interfaces (interfaces.py Protocol/ABC classes in the backend, interfaces.ts in the frontend, or the Next.js route contract for UI pages) and hides its implementation modules.Architectural decisions and rationale:1. Layered architecture instead of microservices or pipe-and-filter: the system has one small team, one dataset family and one request/response workflow (select matchup -> rank -> explain -> export). Layers give a clear separation of concerns and a strict dependency direction without the operational cost of many services; the offline data pipeline (backend/data/etl) is kept outside the request path, so the online system only serves pre-built, cached aggregates and stored ML predictions.2. Recommendation logic lives only in the Domain layer (Baseline Recommendation, Context ML Recommendation). The baseline recommender is the explainable trust anchor; the context ML recommender produces adjusted rankings and reports an explicit error if its model data is unavailable, so ML can be changed without touching the UI or the API.3. The API Coordination subsystem is the single entry point (Facade) for the UI: FastAPI routers validate inputs, enforce roles through dependency injection (Depends(require_auth / require_role)) and delegate to domain and infrastructure subsystems.4. Security is centralised in Access Control Services (role -> resource permission table) and External Integrations (Supabase JWT verification), and mirrored in the Next.js middleware for page routes.5. Infrastructure subsystems isolate technical concerns: Data Access (parquet loading, fingerprint-based cache, Repository-style accessors), Model Management (cross-validated model evaluation), Visualization and Export (SportyPy renderers, PDF builder), External Integrations (Adapter for Supabase and spaCy NLP), Frontend Infrastructure (ApiClient remote proxy over HTTP/JSON).6. Frontend relaxed layering: UI components may call pure frontend domain helpers (app/domain) and the Supabase auth gateway directly; all backend data access goes through the app/services facades and the ApiClient.Quality attributes supported: maintainability and testability (subsystem interfaces, one module per concern), performance (cached aggregates, precomputed predictions, lru/disk caches), security (JWT + role checks at both tiers), explainability (rationale produced next to every ranking) and explicit failure handling (Context reports unavailable model data; the separate baseline page remains available).Note: the offline ETL scripts in backend/data/etl (build_pbp_pipeline, pbp_phase2_build, run_ml_evaluation) are build-time tooling and are not part of the runtime layers. The root Next.js shell (app/layout.tsx, app/page.tsx home landing) is presentation-only and is not modelled as a subsystem.

2.2.2 Layer and Subsystem Dependencies



Dependency (USES) structure between Design Model subsystems, derived from the import statements of the NBAPlayRanker-1 code base (backend Python packages and Next.js app/ modules). Each arrow means the client subsystem imports/calls the supplier subsystem.

2.3 Layers and Subsystems

2.3.1 User Interface Layer

Next.js and React display the coach and analyst workspaces. MatchupDashboardClient holds the selected season, teams and possession cutoff. UsageEfficiencyChart and OpponentDefenseChart render accessible charts with exact values in tables. Changing filters removes the previous results immediately.
Subsystems: the Next.js pages and components the coach and analyst work with. Data Explorer UI shows the cleaned datasets, Recommendation UI holds the Matchup and Context pages, Analytics UI holds the evidence pages, Authentication UI handles sign-in and role-aware navigation, Gameplan UI holds the tactics board and Shot Analysis UI holds the Dataset 2 shot pages. None of these classes calls the backend directly; they go through Frontend Services.

2.3.2 Application Layer

FastAPI coordinates requests, checks identity and permissions, validates filters and invokes the domain service. GET /analytics/playtypes supplies both Alpha views from one consistent response. Invalid inputs, unavailable data and expired sessions return clear errors rather than partial comparisons.
Subsystems: API Coordination receives every request and routes it, Recommendation Services and Analytics Services coordinate the domain work for each page, Access Control Services checks the session and role, and Frontend Services is the browser-side facade that each page area calls. This layer contains no basketball logic, which keeps the rules in one place.

2.3.3 Domain Layer

The baseline service aggregates regular-season play-type records. The dashboard calculates usage, points per possession and differences from the league for the same play type. The existing baseline recommender blends team offense with opponent points allowed. Model and shot services remain separate from the new descriptive dashboard.
Subsystems: Baseline Recommendation and Context ML Recommendation hold the ranking logic, Statistical Analysis holds the model evidence, Shot Analysis holds the play-by-play shot logic and Frontend Domain holds small pure rules used by the pages, such as the context overlay weights and the top pick rule.

2.3.4 Infrastructure Layer

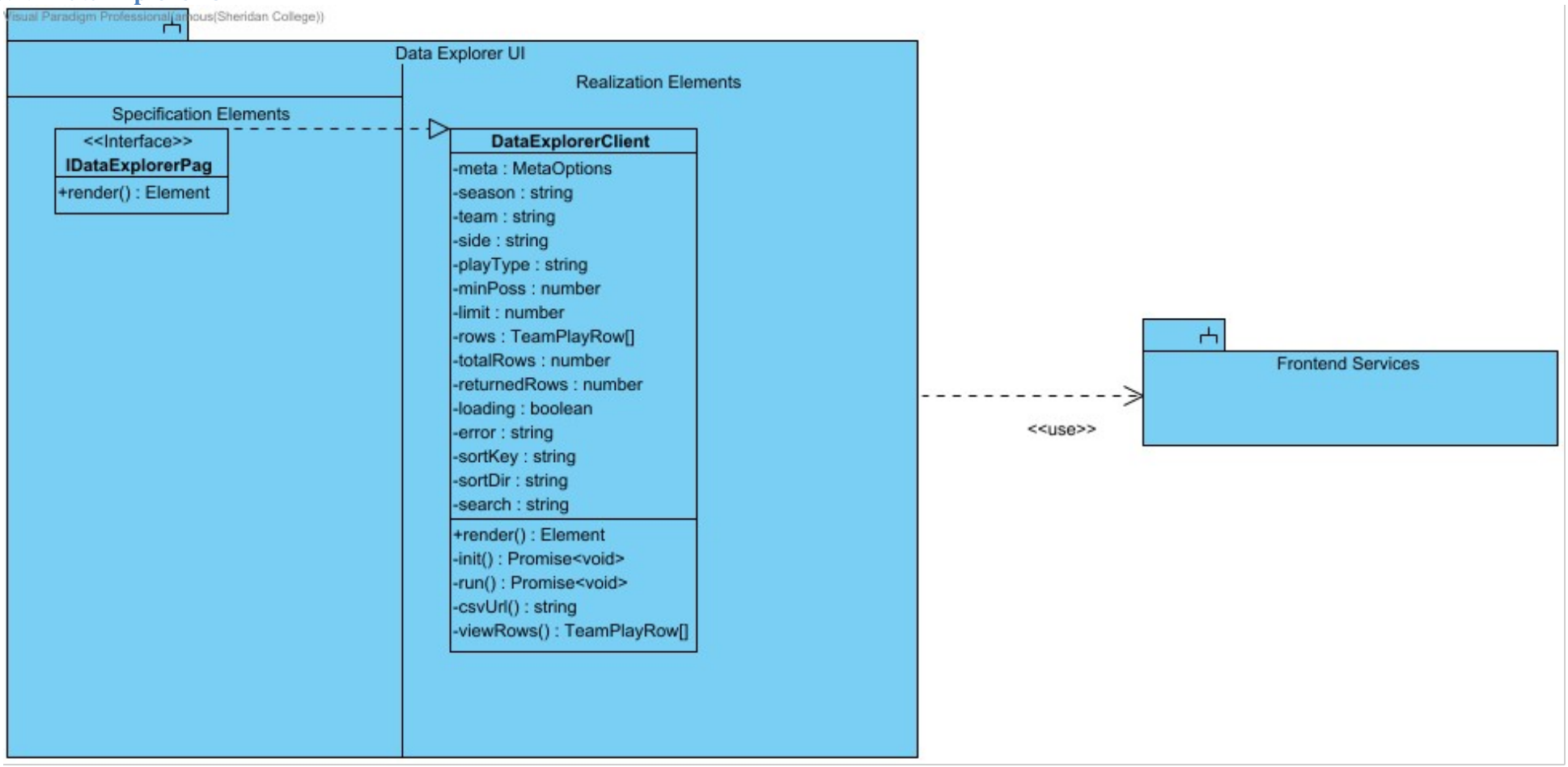
Data access loads versioned source files and validated cached artifacts. The frontend request adapter attaches the current session token, validates the response and cancels obsolete requests. Export services handle existing CSV, JSON and PDF reports. Source and aggregation versions identify the inputs behind saved predictions.
Subsystems: Data Access reads and caches the Synergy and play-by-play files, Model Management loads and evaluates the trained models, Visualization and Export builds images and files, External Integrations wraps Supabase and the NLP library, and Frontend Infrastructure holds the typed HTTP clients and the Supabase gateway used by the browser.

3. Detailed Design

This section goes through each layer and its subsystems. Every subsystem has an overview class diagram that shows its public classes with their main fields and methods, the internal classes that support them, and the subsystems it depends on. Where the model has a detailed design (elaboration) class diagram or a detailed sequence diagram for part of a subsystem, it is shown under that subsystem.

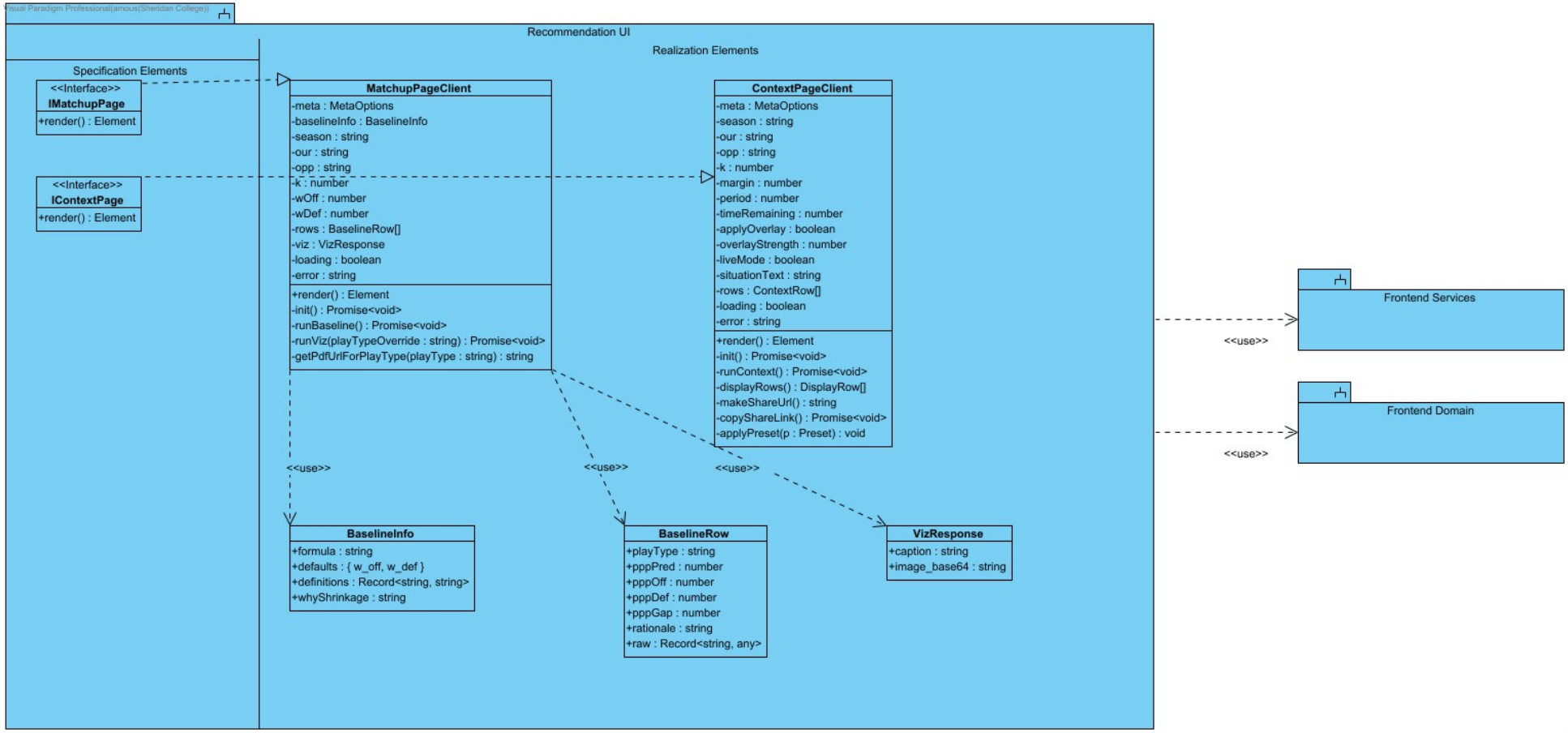
3.1 User Interface Layer

3.1.1 Data Explorer UI



Analyst workspace for browsing and filtering the cleaned Synergy team play-type table and exporting it. Code: app/data-explorer/page.tsx, app/ui/data-explorer/DataExplorerClient.tsx. Uses Frontend Services (app/services/dataExplorer.ts).

3.1.2 Recommendation UI



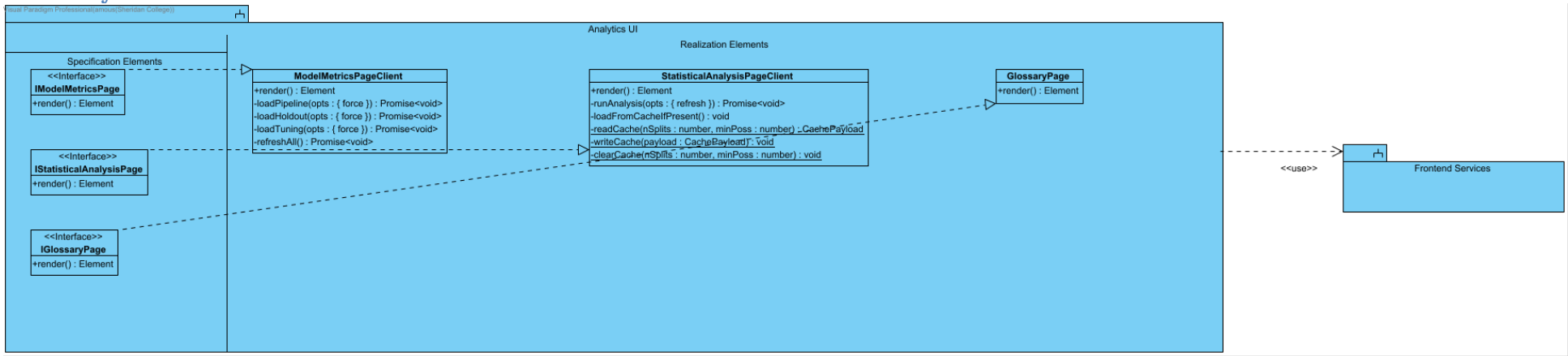
Coach-facing recommendation pages: baseline matchup ranking (/matchup) and context-aware ML ranking (/context). Code: app/matchup, app/context, app/ui/recommendation/_components. Uses Frontend Services (app/services/recommendation.ts) and Frontend Domain helpers.

3.1.2.1 Wireflow - Baseline Comparisons



SCRUM-695/696. Native conceptual screen-state design for the implemented coach /matchup-dashboard. These are model views, not captured application screenshots. Valid filters automatically load both views; switching views needs no new request. Beta and Final requirements remain planned.

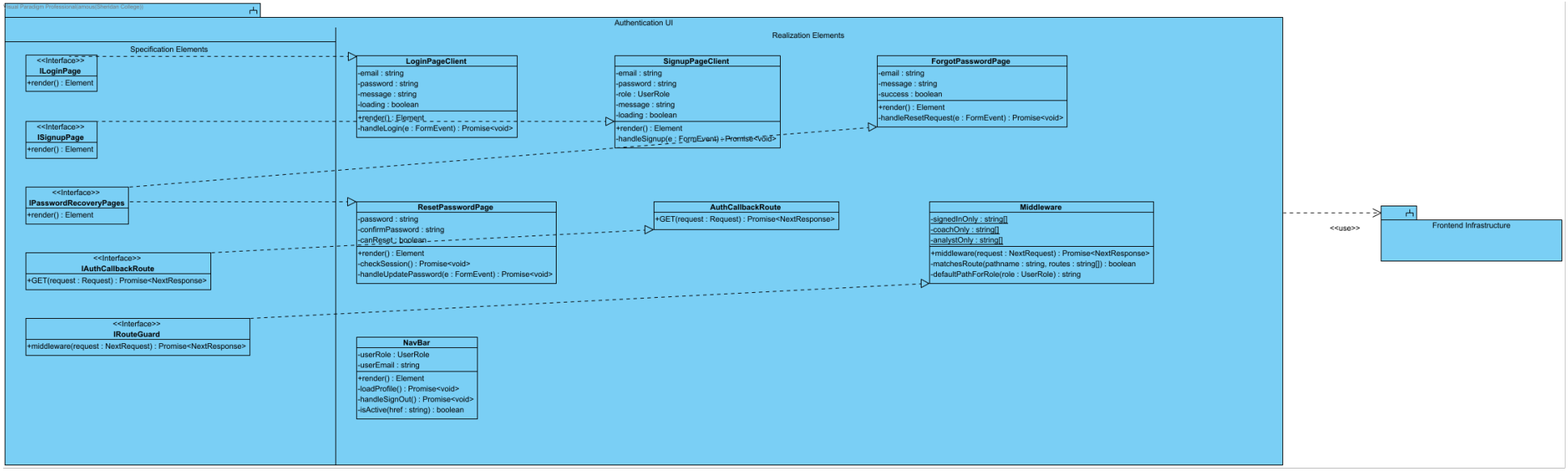
3.1.3 Analytics UI



Evidence pages for the play-type models: model metrics (/model-metrics), statistical analysis (/statistical-analysis) and the glossary (/glossary). Code: app/ui/analytics/_components, app/glossary/page.tsx. Uses Frontend Services (app/services/analytics.ts).

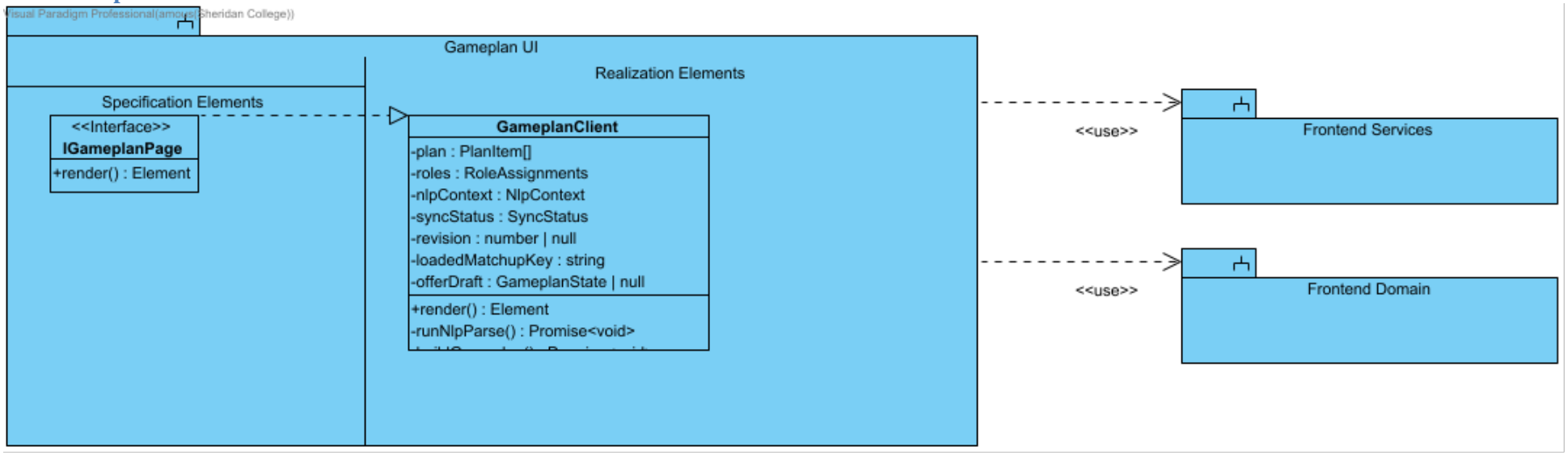
SCRUM-476. Expanding-window held-out-season PPP diagnostics using fixed Ridge and training-only StandardScaler/reliability normalization. Report signed bias, MAE/RMSE, continuous prediction bins, sparse-bin warnings and fold provenance. Same-season explanatory features mean retrospective evaluation, not pre-game forecasting proof.

3.1.4 Authentication UI



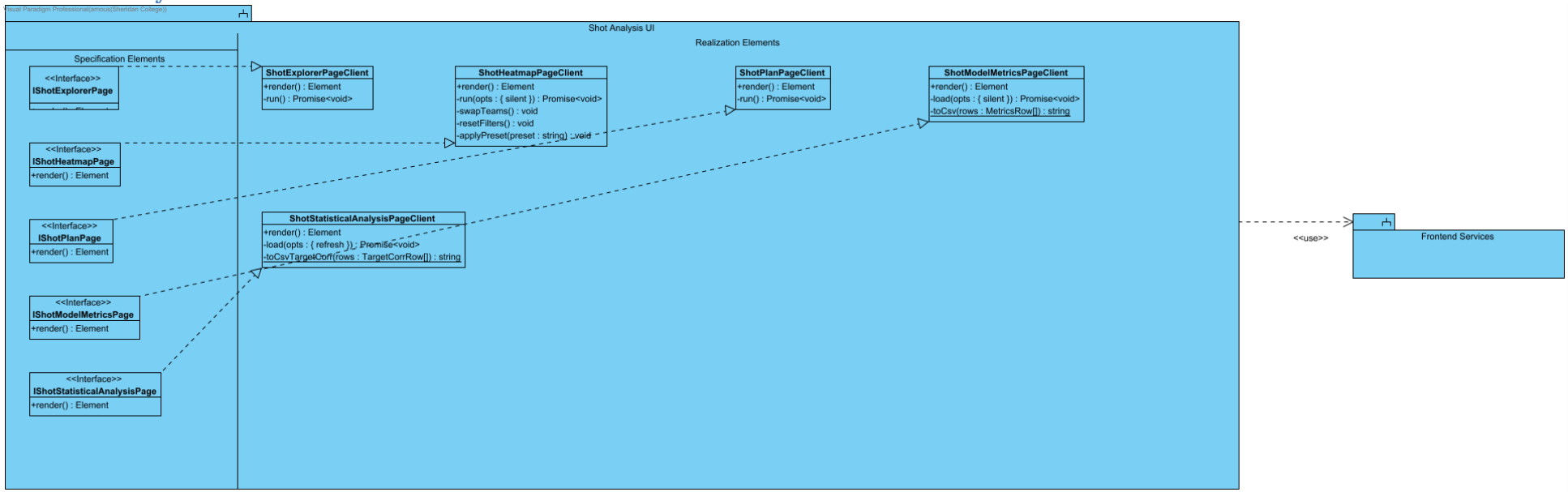
Sign-in, sign-up and password-recovery screens, the Supabase auth callback route, the role-aware navigation bar and the Next.js middleware that guards coach-only and analyst-only routes. Code: app/login, app/signup, app/forgot-password, app/reset-password, app/auth/callback, app/ui/authentication, app/ui/_shared/components/NavBar.tsx, middleware.ts. Uses Frontend Infrastructure (AuthGateway, SupabaseClient).

3.1.5 Gameplan UI



Coach gameplan flow (/gameplan): NLP situation parsing, smart + baseline recommendations, plan builder and explanations. Code: app/gameplan/page.tsx, app/ui/gameplan/_components/GameplanClient.tsx. Uses Frontend Services (recommendation, shotAnalysis, gameplan) and Frontend Domain (coaching-knowledge).

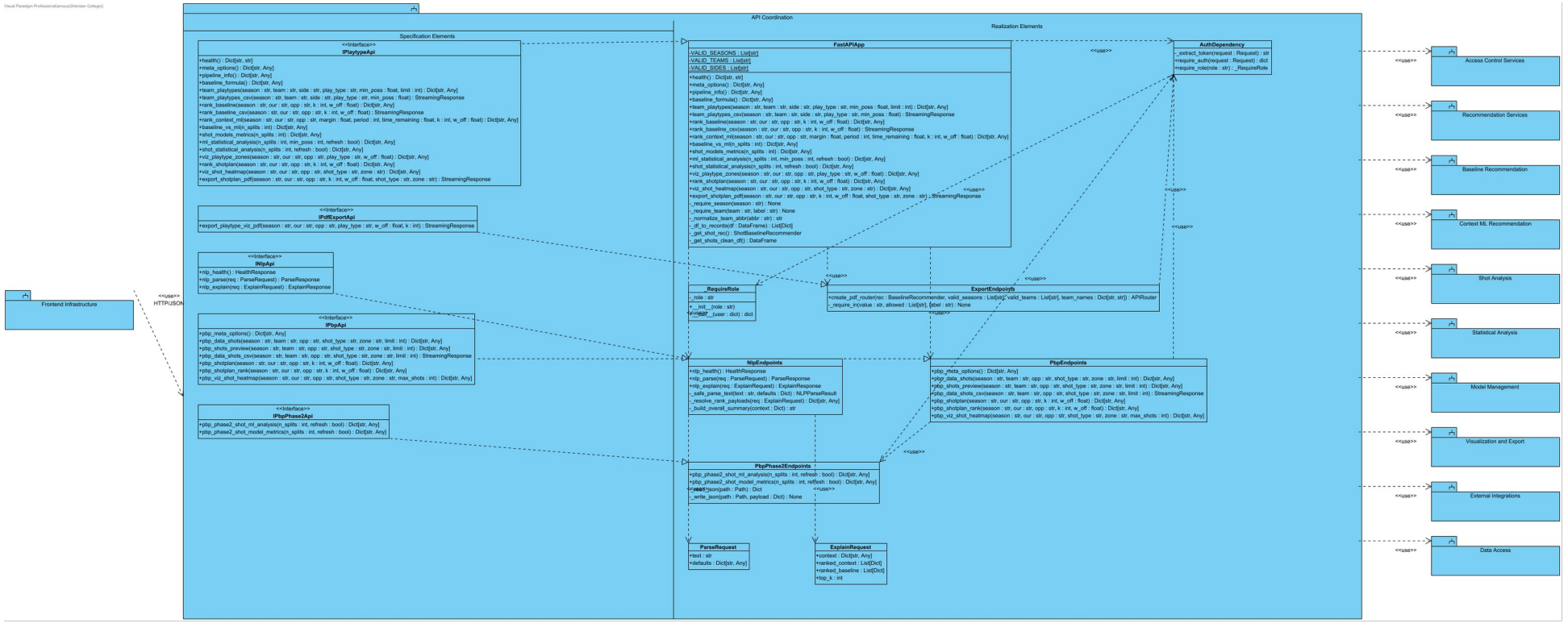
3.1.6 Shot Analysis UI



Dataset 2 (play-by-play shots) pages: shot explorer, shot heatmap, shot plan, shot model metrics and shot statistical analysis. Code: app/shot-*/page.tsx, app/ui/shot-analysis/_components. Uses Frontend Services (app/services/shotAnalysis.ts).

3.2 Application Layer

3.2.1 API Coordination

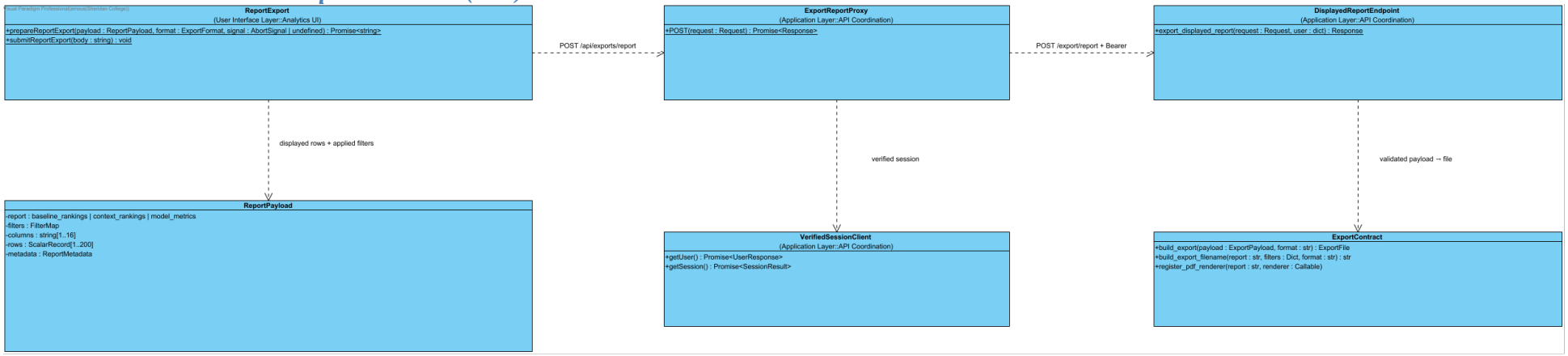


FastAPI entry point of the backend. Routers validate query parameters, enforce authentication/roles through FastAPI dependencies and delegate to application, domain and infrastructure subsystems. Code: backend/application/api_coordination/*.py.

SCRUM-475. Historical modeled Top-K comparison, not causal or held-out performance. Team baseline uses SUM(PTS)/SUM(POSS) over all team-season offensive rows. Selected modeled PPP uses SUM(PPP_PRED*POSS_OFF)/SUM(POSS_OFF). JSON and CSV contain scope, numerators, denominators, K, units, formulas, source SHA-256 and row contributions. Missing PTS fails explicitly; zero baseline gives null relative uplift. SCRUM-474 remains a separate evaluation dependency.

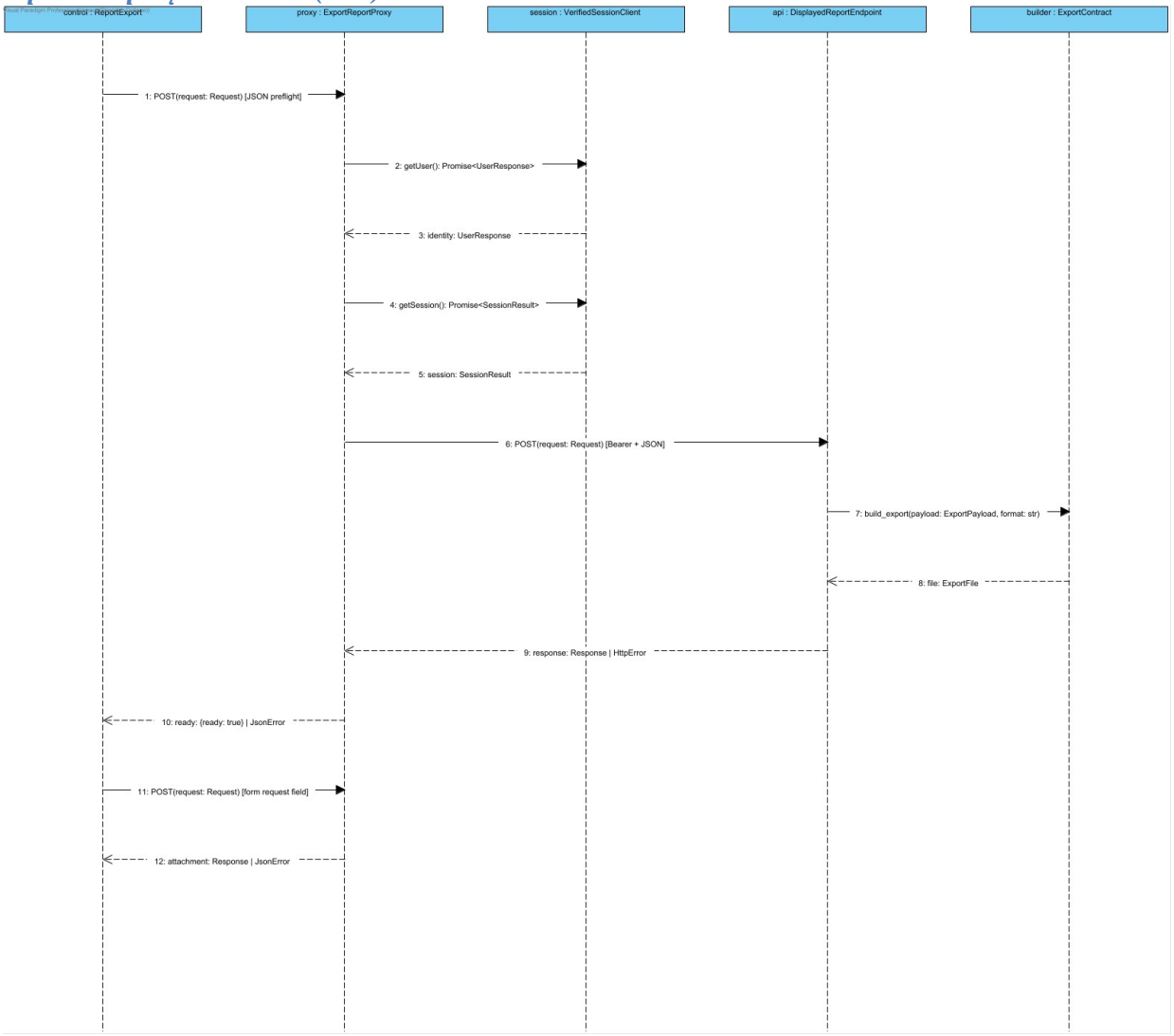
SCRUM-476. Expanding-window held-out-season PPP diagnostics using fixed Ridge and training-only StandardScaler/reliability normalization. Report signed bias, MAE/RMSE, continuous prediction bins, sparse-bin warnings and fold provenance. Same-season explanatory features mean retrospective evaluation, not pre-game forecasting proof.

3.2.1.1 Elaboration - Shared Export Controls (486)



SCRUM-486. One Export control on Matchup, Context and Model Metrics offers CSV, JSON and PDF for the displayed result. Capture rows and the filters that produced them; disable export when results are empty, loading or no longer match the controls. This is a displayed-result snapshot, not a server recomputation. POST /api/exports/report validates the cookie session and forwards Bearer credentials to the fixed POST /export/report backend. Backend enforces report roles, 256 KiB, 200 rows, 16 columns and finite scalar cells. JSON records export_source=displayed_result_snapshot; CSV neutralizes spreadsheet formulas; PDF includes title, filters, limitations and table. Errors remain JSON without an attachment header.

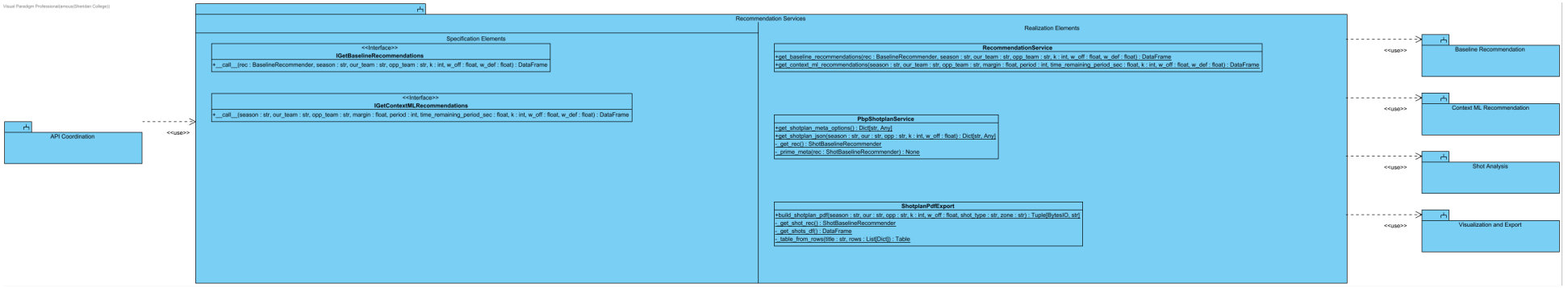
3.2.1.2 Detailed Sequence - Export Displayed Results (486)



SCRUM-486. One Export control on Matchup, Context and Model Metrics offers CSV, JSON and PDF for the displayed result. Capture rows and the filters that produced them; disable export when results are empty, loading or no longer match the controls. This is a displayed-result snapshot, not a server recomputation. POST /api/exports/report validates the cookie session and forwards Bearer credentials to the fixed POST /export/report backend. Backend enforces report roles,

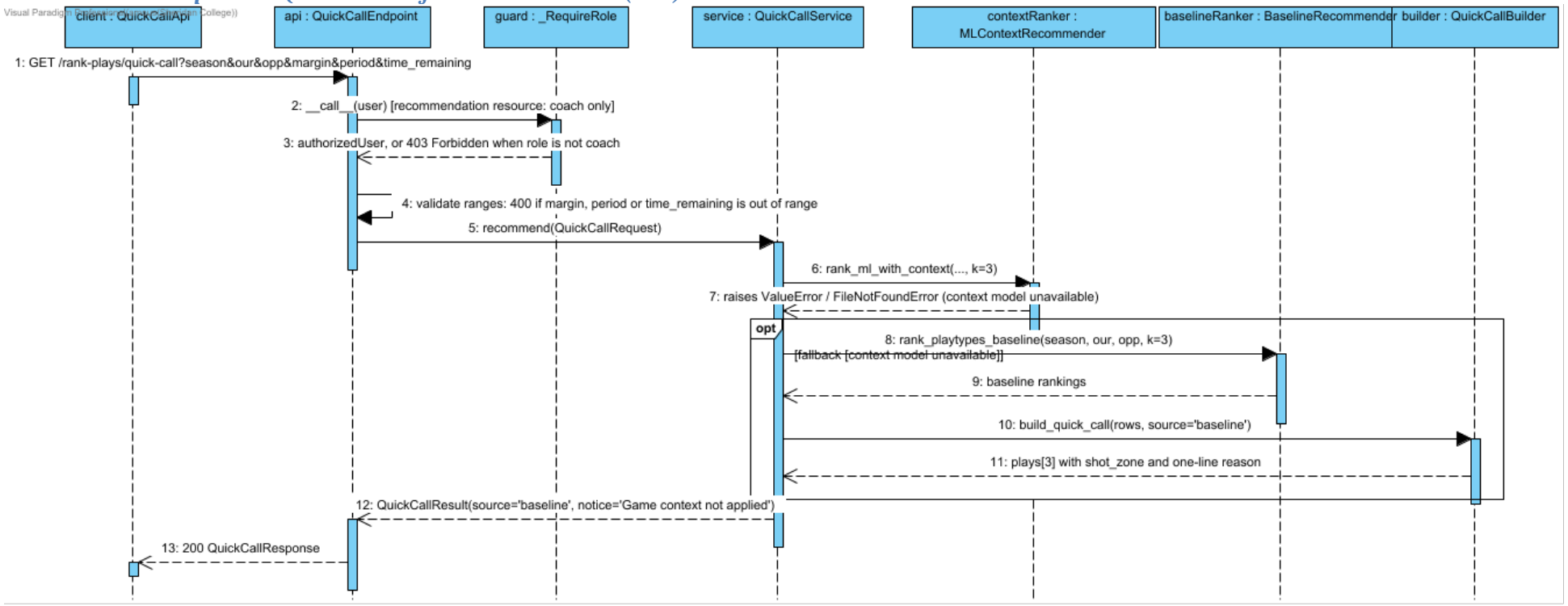
256 KiB, 200 rows, 16 columns and finite scalar cells. JSON records export_source=displayed_result_snapshot; CSV neutralizes spreadsheet formulas; PDF includes title, filters, limitations and table. Errors remain JSON without an attachment header.
Alternatives: stale/loading/empty disables Export. Invalid session401; wrong report role403; malformed/oversized body400/413/422; upstream failure502. Preflight failure displays Retry; successful preflight starts native same-origin form download. No token is placed in a URL.
prepareReportExport(payload: ReportPayload, format: ExportFormat, signal?: AbortSignal): Promise<string> returns the encoded body. submitReportExport(body: string): void starts the native form request. Both requests run the same server checks. UserResponse and SessionResult are result envelopes, not request parameters. Errors return JSON with no attachment header. Pipeline/model values are never recomputed during export.

3.2.2 Recommendation Services



Application workflows for recommendations: baseline and context-ML ranking facades, the Dataset 2 shot-plan workflow and the shot-plan PDF export. Code: backend/application/recommendation_services/*.py.

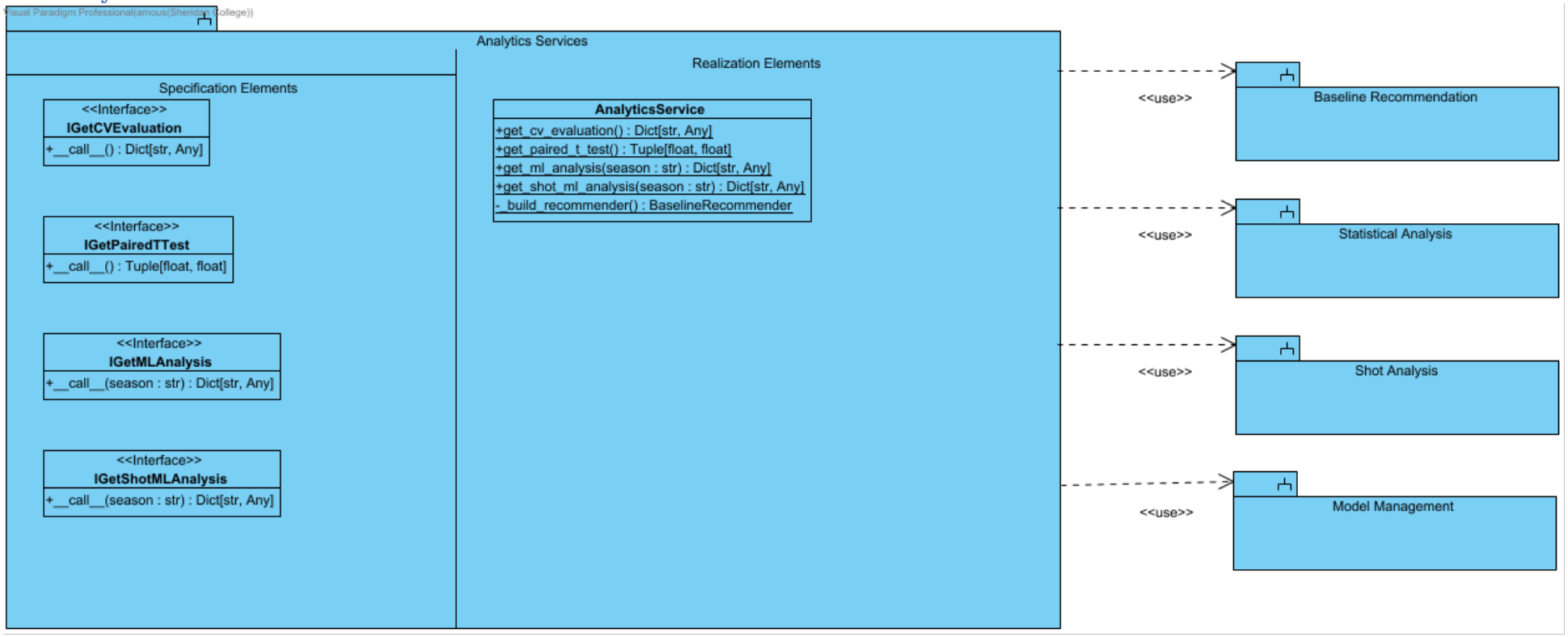
3.2.2.1 Detailed Sequence - Quick-Call Refresh and Fallback (628)



Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.
Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.
Implementation note: GameplanClient is the current /nlp/parse caller. The revised Build resolves a single situation snapshot, awaits parsing, validates supported conditions and sends that snapshot to contextRank without waiting for React state. Period 5 remains overtime with a 300-second clock. Unsupported instructions

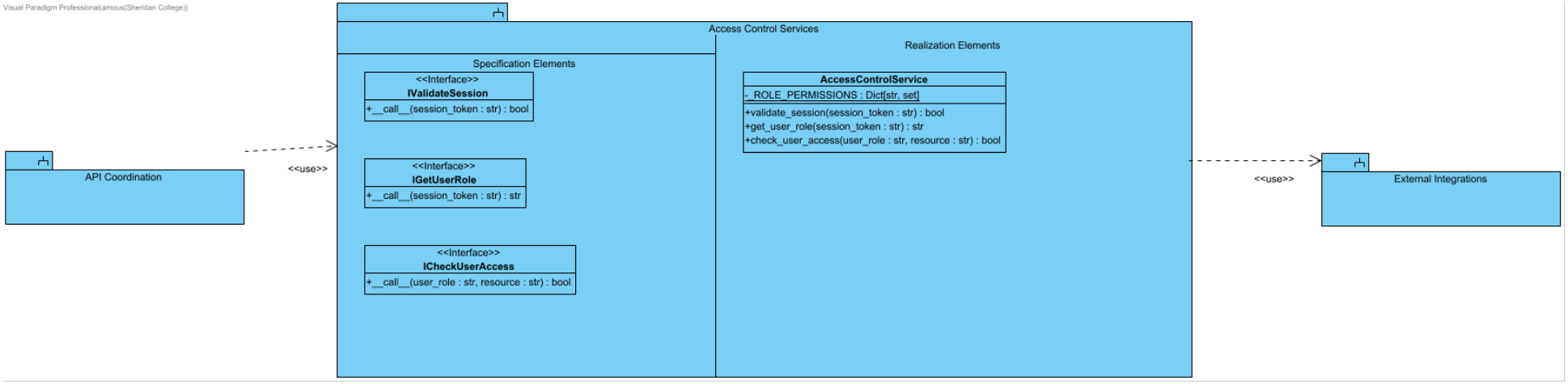
remain visible as notes rather than silently changing the ranking. This sequence shows the parse subflow; the rank request is documented in Behavioural Overview - Rank Plays with Game Context. ParseRequest={text:string,defaults?:dict}. ParseResponse and ContextGuardResult fields are defined on the named response messages.

3.2.3 Analytics Services



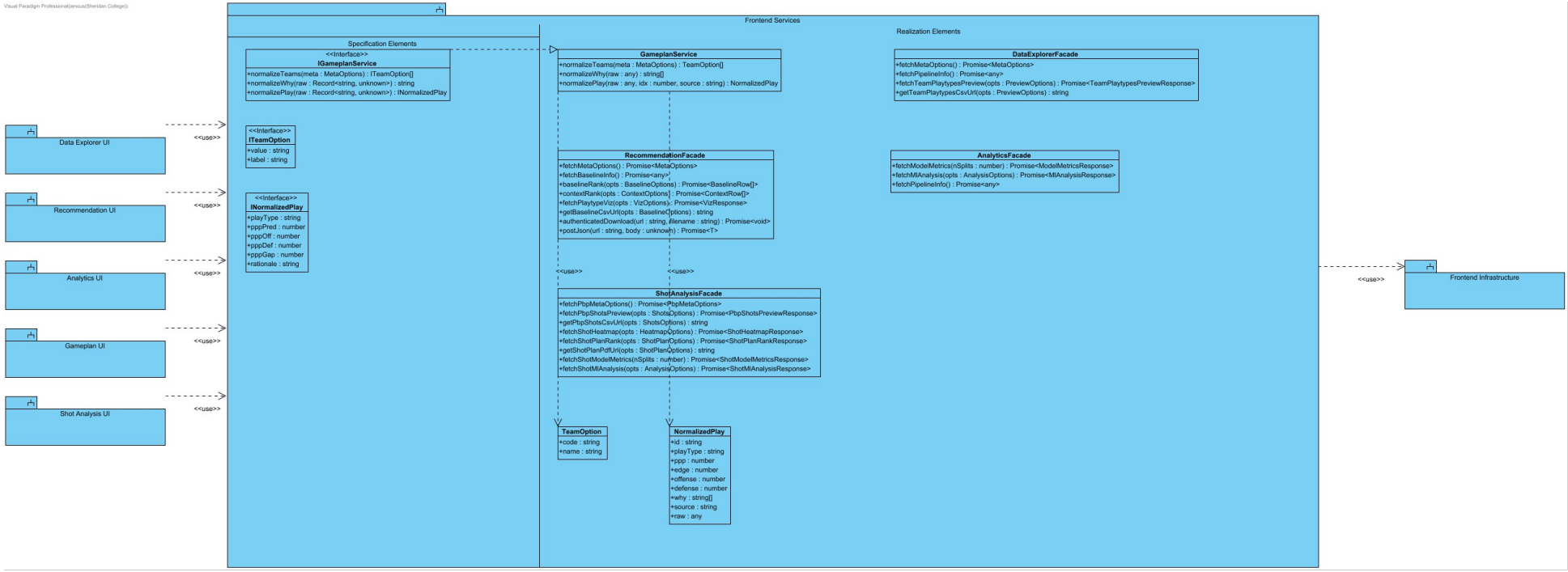
Application workflows for model evaluation and statistical analysis evidence. Code: backend/application/analytics_services/*.py.

3.2.4 Access Control Services



Role-based access control: session validation, role lookup from JWT claims and the resource -> role permission table. Code: backend/application/access_control_services/*.py.

3.2.5 Frontend Services



Frontend application-service layer (app/services): facades that expose the API client functions each UI area may use, and the gameplan normalisation service. UI components import only from here for backend data.

3.3 Domain Layer

The domain classes follow these calculation rules for the baseline comparison views:

Compare the same regular season across all displayed teams and league references. Do not mix playoff records into regular-season totals.

Calculate team PPP as total points divided by possessions. Calculate usage as play-type possessions divided by the team's recorded offensive play-type possessions, multiplied by 100.

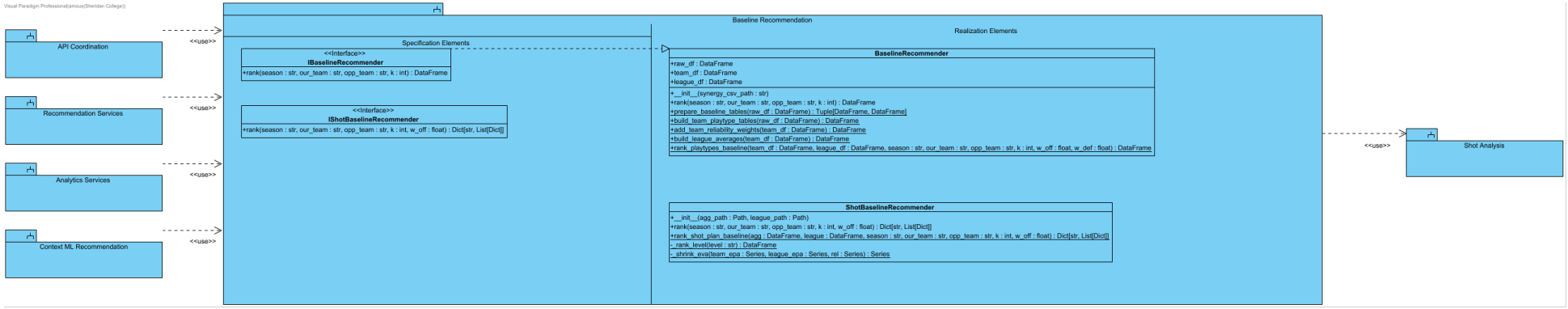
Calculate league values from pooled counts for the same season, side and play type. Calculate FG percentage from field goals made and attempted; weight source effective field-goal rates by attempts.

For the usage chart, subtract the same play type's league usage and PPP from the team's values. A high-efficiency, low-usage pattern is a review prompt, not proof that using that play more will improve results.

For opponent weak spots, subtract league allowed PPP from the opponent's allowed PPP. Rank the differences from highest to lowest. Highlight at most three positive differences that meet the possession cutoff.

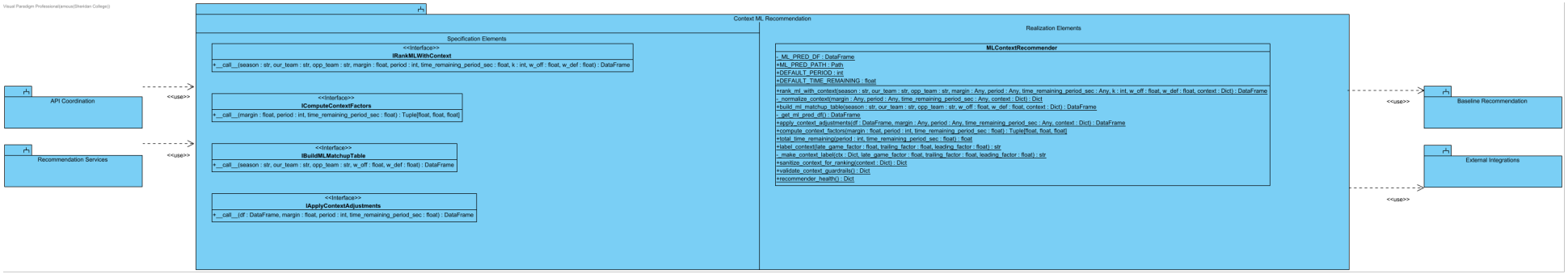
The possession cutoff marks small samples without removing rows or changing totals. Small samples cannot receive a weak-spot highlight. The cutoff is a display rule, not a confidence interval.
Missing defensive categories remain unavailable. They must not appear as zero points allowed. The current source contains eleven offensive categories and seven defensive categories.
These comparisons use whole-season profiles. Selecting two teams does not restrict the data to games played against each other.
Only approved coaches can use the matchup dashboard. Validate the current profile role on the API as well as the page route.
Discard results that no longer match the selected filters. On an error, remove the stale chart and provide a retry action.
Keep source checks, descriptive comparisons, retrospective model diagnostics and prospective evaluation distinct. Observed-season model performance does not establish pre-game forecast accuracy.

3.3.1 Baseline Recommendation



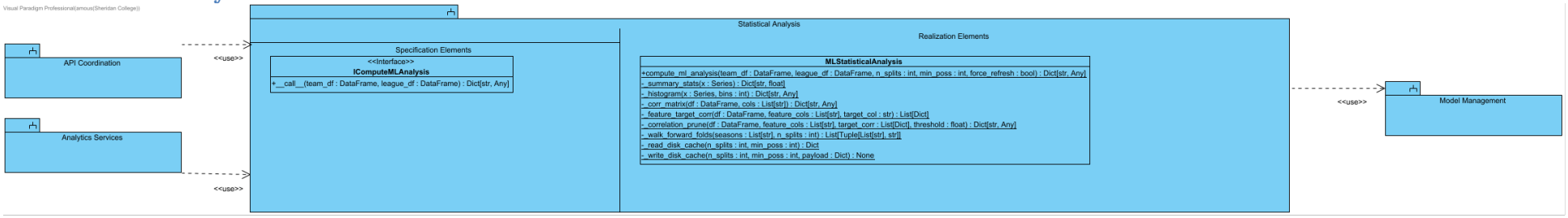
Explainable baseline ranking: possession-weighted team play-type tables, reliability (shrinkage) weights, league averages and the offence/defence blended PPP ranking; plus the Dataset 2 shot-plan baseline. Code: backend/domain/baseline_recommendation/*.py.

3.3.2 Context ML Recommendation



Context-aware re-ranking: blends stored ML PPP predictions with the baseline, applies late-game / score-margin / NLP context adjustments and builds the rationale. Code: backend/domain/context_ml_recommendation/*.py.

3.3.3 Statistical Analysis

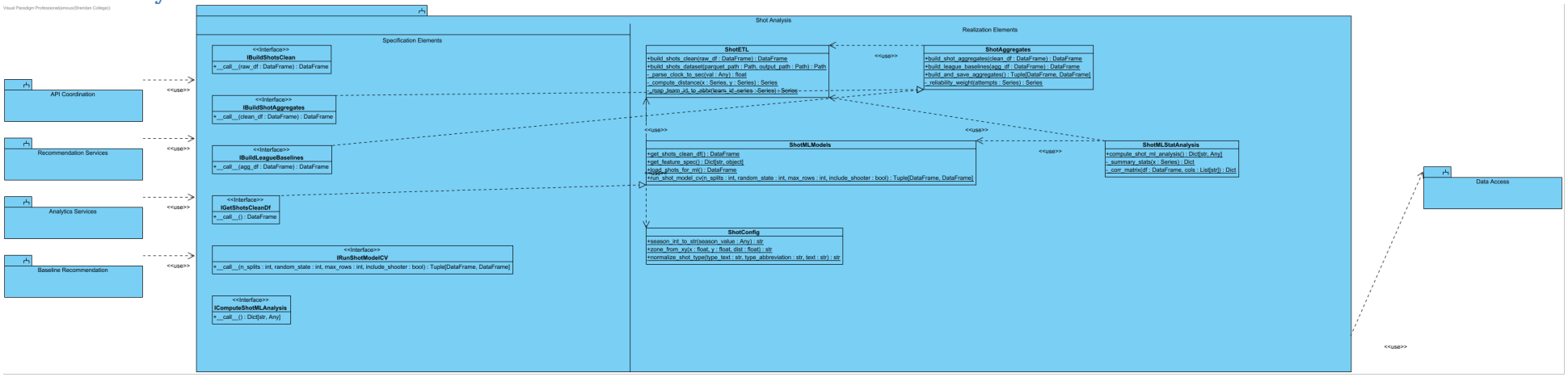


Statistical evidence for the play-type ML pipeline: EDA summaries, histograms, correlation matrix, feature-target correlation, correlation pruning, walk-forward folds and model selection (cached to disk). Code: backend/domain/statistical_analysis/*.py.

SCRUM-475. Historical modeled Top-K comparison, not causal or held-out performance. Team baseline uses SUM(PTS)/SUM(POSS) over all team-season offensive rows. Selected modeled PPP uses SUM(PPP_PRED*POSS_OFF)/SUM(POSS_OFF). JSON and CSV contain scope, numerators, denominators, K, units, formulas, source SHA-256 and row contributions. Missing PTS fails explicitly; zero baseline gives null relative uplift. SCRUM-474 remains a separate evaluation dependency.

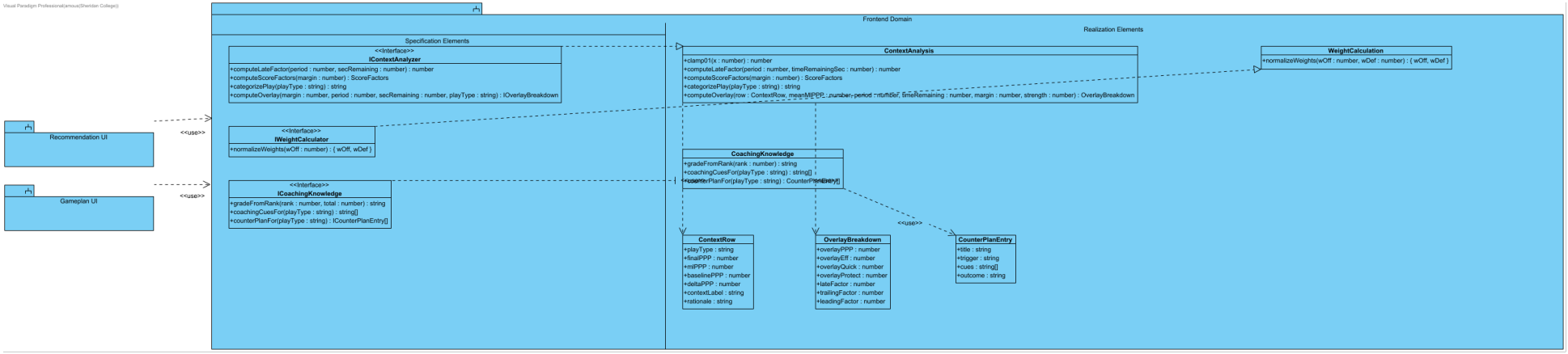
SCRUM-476. Expanding-window held-out-season PPP diagnostics using fixed Ridge and training-only StandardScaler/reliability normalization. Report signed bias, MAE/RMSE, continuous prediction bins, sparse-bin warnings and fold provenance. Same-season explanatory features mean retrospective evaluation, not pre-game forecasting proof.

3.3.4 Shot Analysis



Dataset 2 (play-by-play shots): cleaning/ETL, shot-type and zone aggregates, league baselines, shot ML model cross-validation and shot statistical analysis. Code: backend/domain/shot_analysis/*.py.

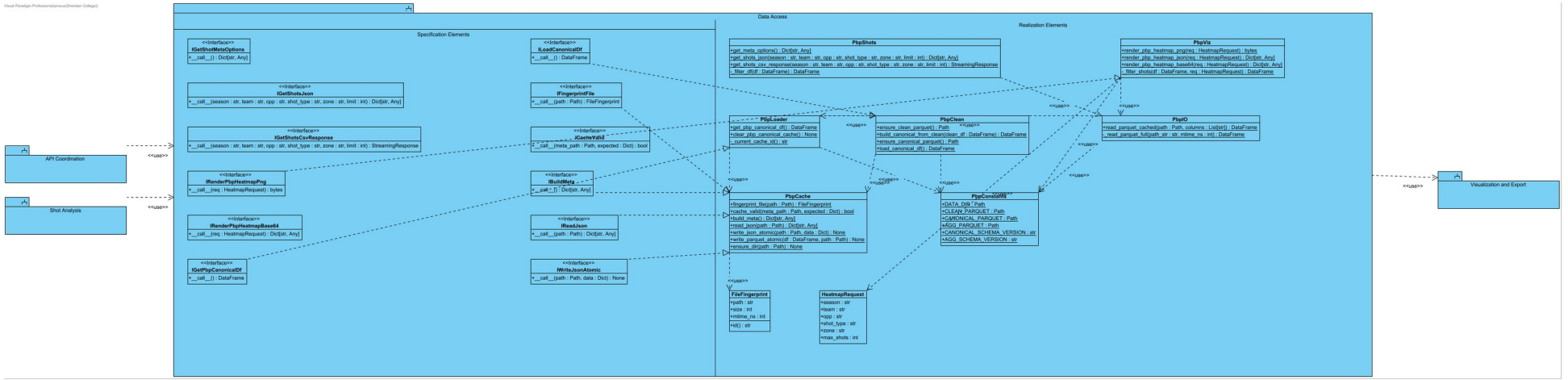
3.3.5 Frontend Domain



Frontend business helpers (app/domain): context overlay scoring for the Context page, offence/defence weight normalisation and coaching knowledge (grades, cues, counter-plans) for the gameplan. Pure functions, no I/O.

3.4 Infrastructure Layer

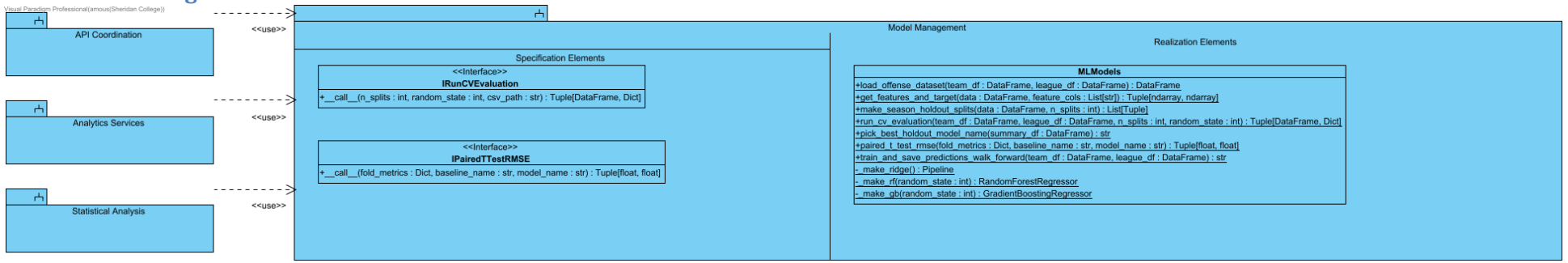
3.4.1 Data Access



Loading and caching of the play-by-play (Dataset 2) parquet files: canonical dataset build, fingerprint-validated cache, cached parquet reads, shot queries/CSV and heatmap requests. Code: backend/infrastructure/data_access/*.py.

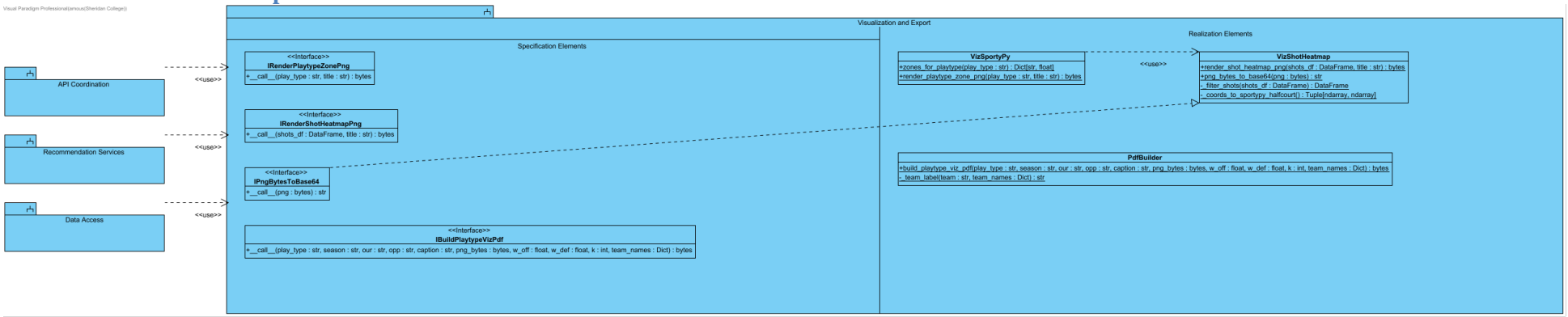
SCRUM-481. public.gameplan_notes composite PK(user_id,season,our_team,opp_team). FK user_id -> auth.users ON DELETE CASCADE. Owner plus current protected profiles.role=coach is required for every CRUD action. Anonymous, analyst and pending users denied. Payload <=65536 bytes. Identity and server audit fields cannot be updated by clients. UI remains localStorage; cloud persistence is future SCRUM-482. Source: supabase/migrations/20261003211500_gameplan_state_schema.sql.

3.4.2 Model Management



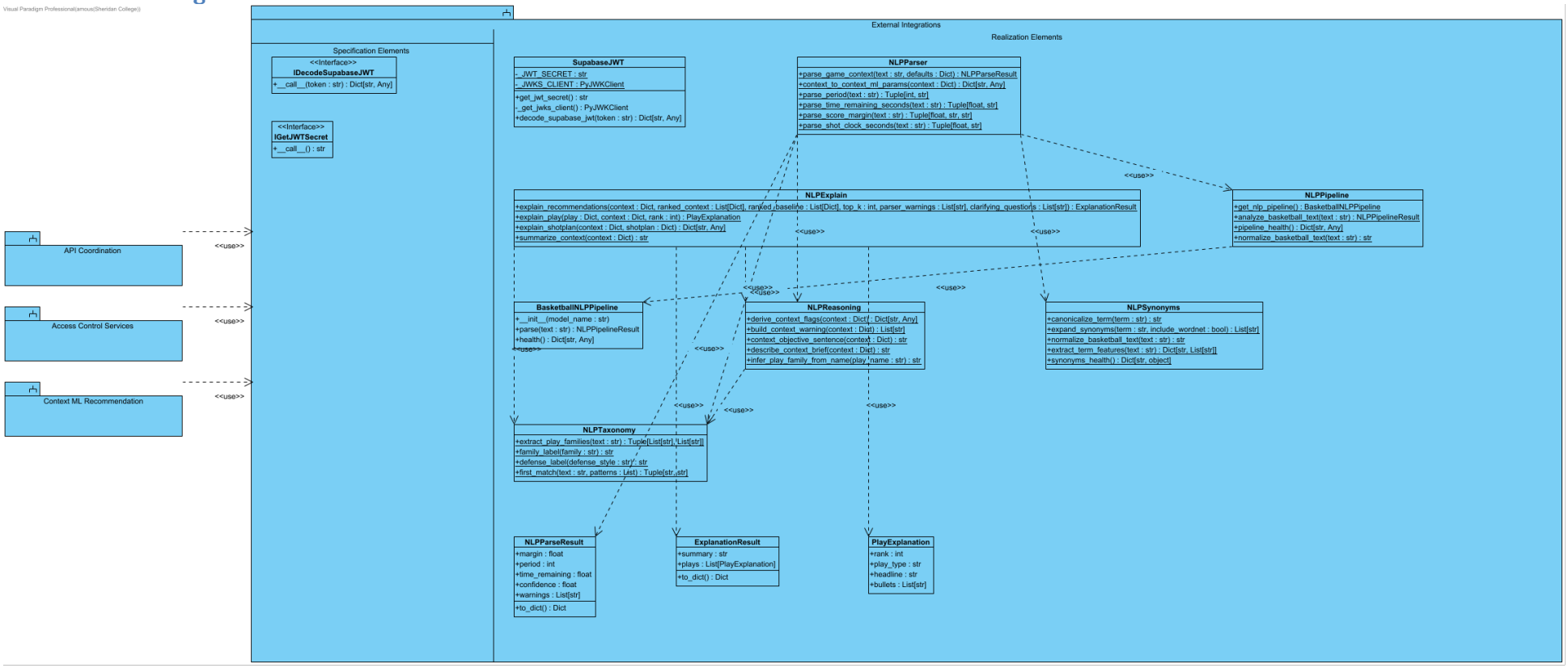
Play-type ML model definitions and evaluation: dataset assembly, season hold-out splits, Ridge / Random Forest / Gradient Boosting cross-validation, paired t-test and walk-forward prediction export. Code: backend/infrastructure/model_management/*.py.

3.4.3 Visualization and Export



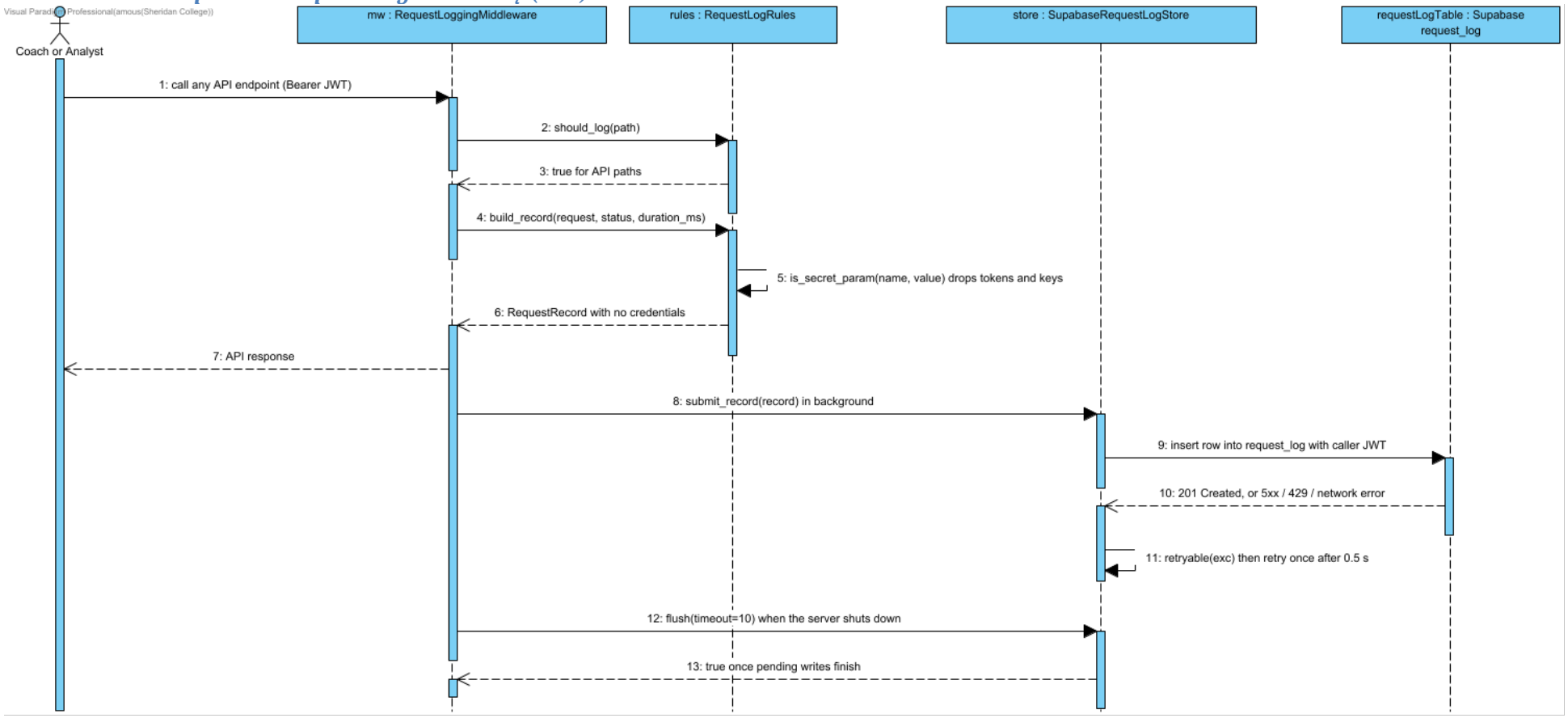
Court visualisations and exports: SportyPy play-type zone PNGs, shot heatmap PNGs (base64 for the UI) and the play-type visualisation PDF. Code: backend/infrastructure/visualization_and_export/*.py.

3.4.4 External Integrations

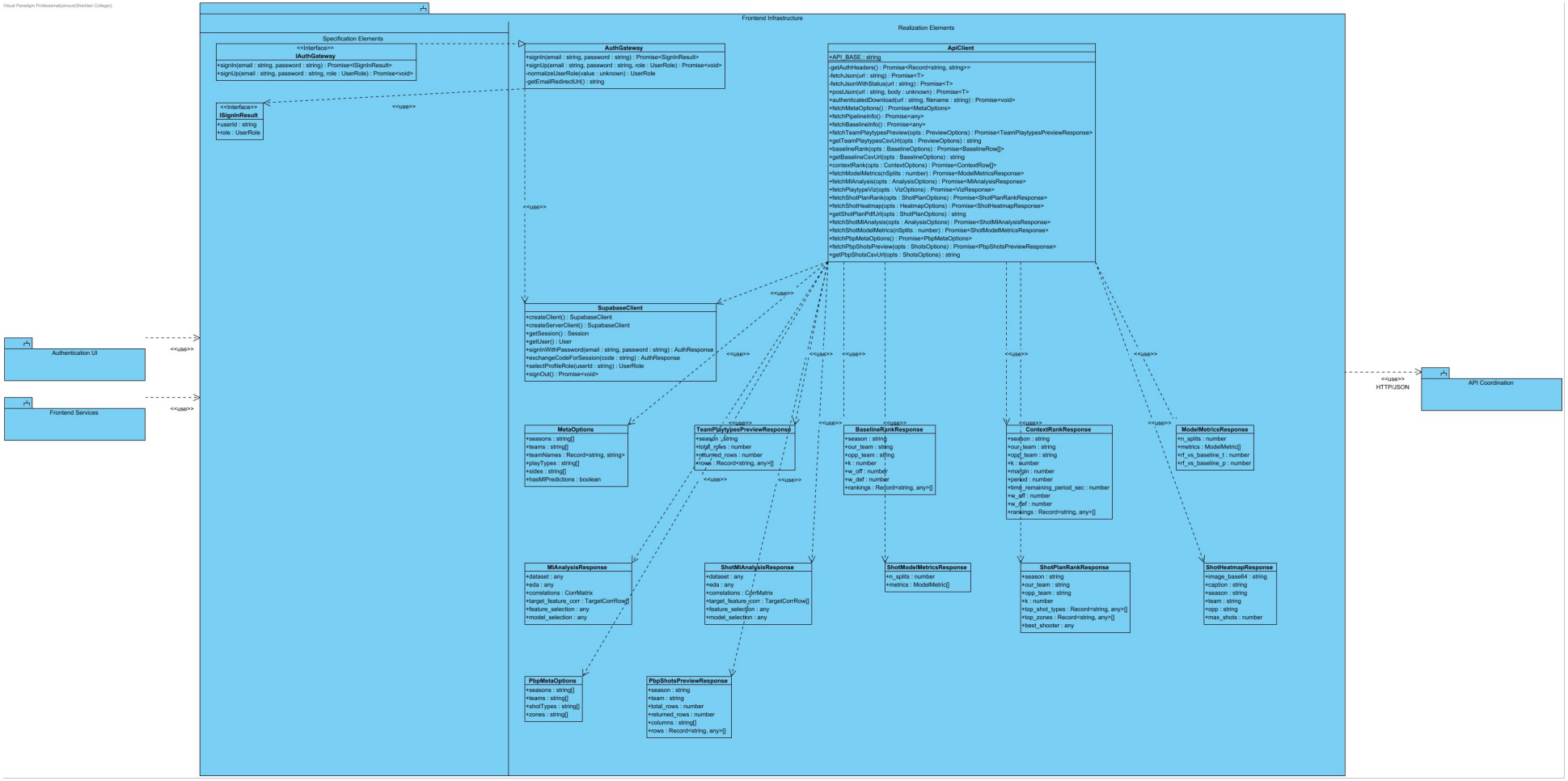


Adapters to external services and libraries: Supabase JWT verification (HS256 secret or JWKS) and the spaCy/WordNet NLP pipeline used to parse game situations and explain recommendations. Code: backend/infrastructure/external_integrations/*.py.

3.4.4.1 Detailed Sequence - Request Log Durability (493)

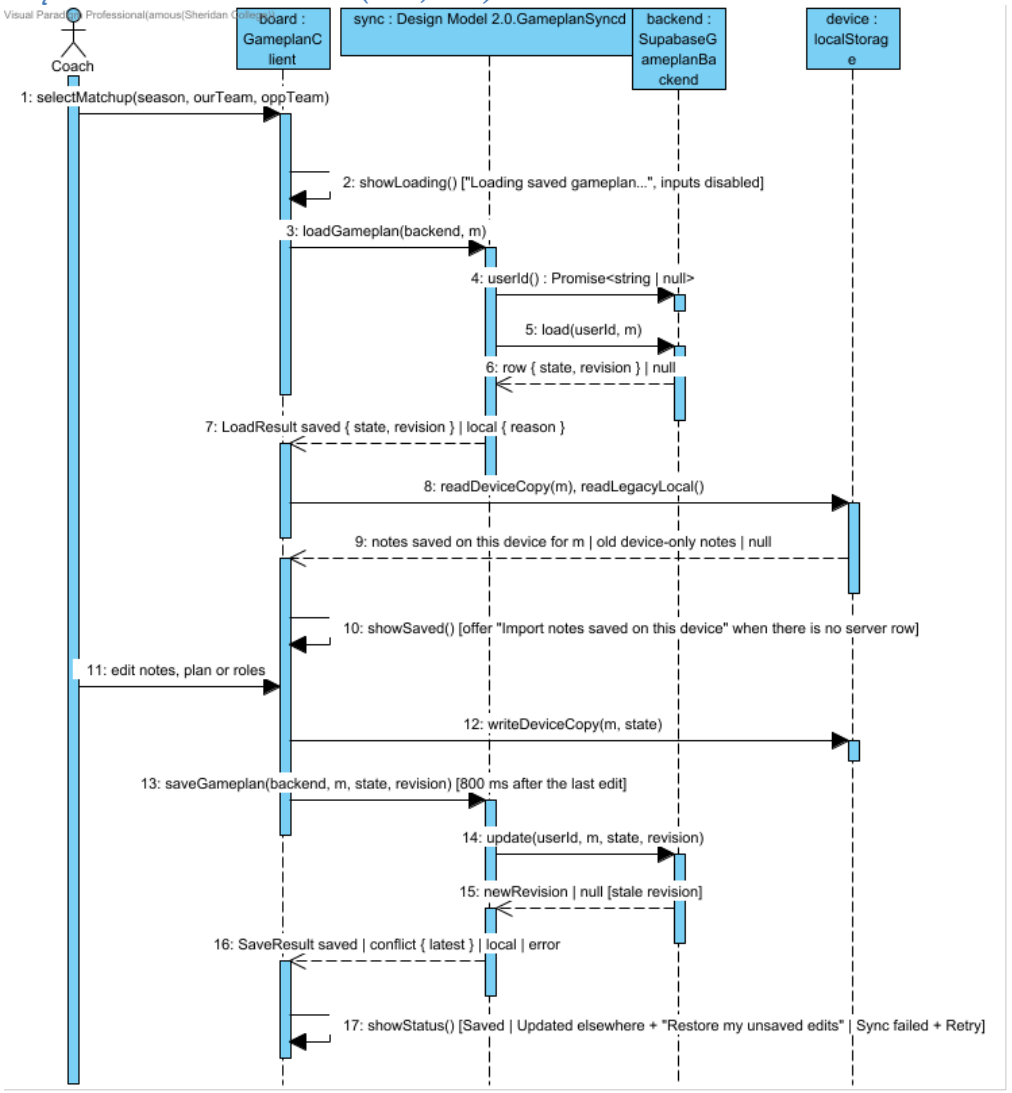


3.4.5 Frontend Infrastructure



Next.js infrastructure: the typed HTTP client for the FastAPI backend (app/infrastructure/api-client.ts) with its response types, the Supabase authentication gateway (app/infrastructure/auth.ts) and the Supabase browser/server clients (lib/supabase).

3.4.5.1 Detailed Sequence - Gameplan Sync and Device Fallback (482, 483)



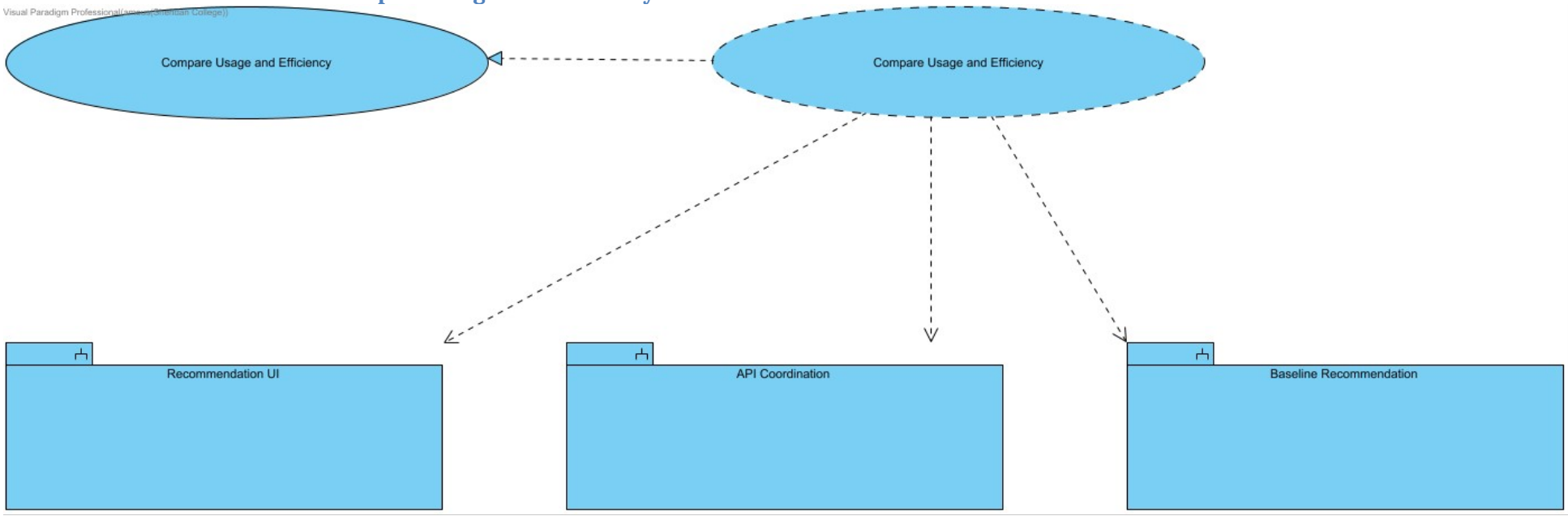
4. Interaction Model

This section lists the interactions of the system. Each interaction realizes a use case from the SRS and is described through a collaboration overview, a structural design (class diagram of the classes that take part, with their main fields and methods) and a behavioural design (sequence diagram). The sequence diagrams show the main flow and the error paths that matter to the user, such as an invalid filter, missing data, an expired session or an unavailable service.

4.1 Compare Usage and Efficiency

Use case A35cd925e814ba68; SCRUM-695. The collaboration reuses the existing application and deployment layers. Numeric fields and alternate outcomes follow the SRS Flow of Events.

4.1.1 Collaboration Overview - Compare Usage and Efficiency



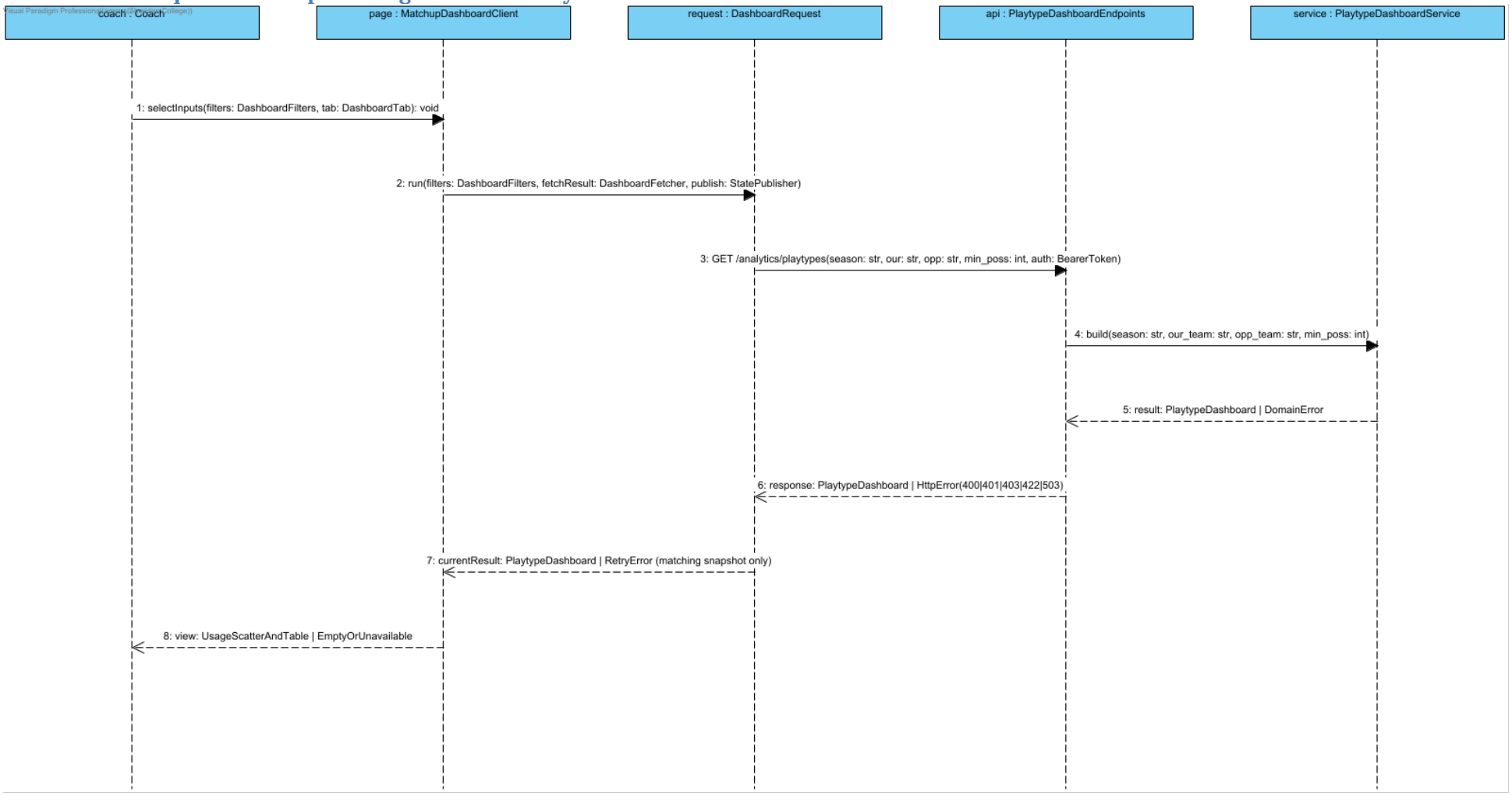
Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

4.1.2 Structural Design - Compare Usage and Efficiency



Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

4.1.3 Detailed Sequence - Compare Usage and Efficiency

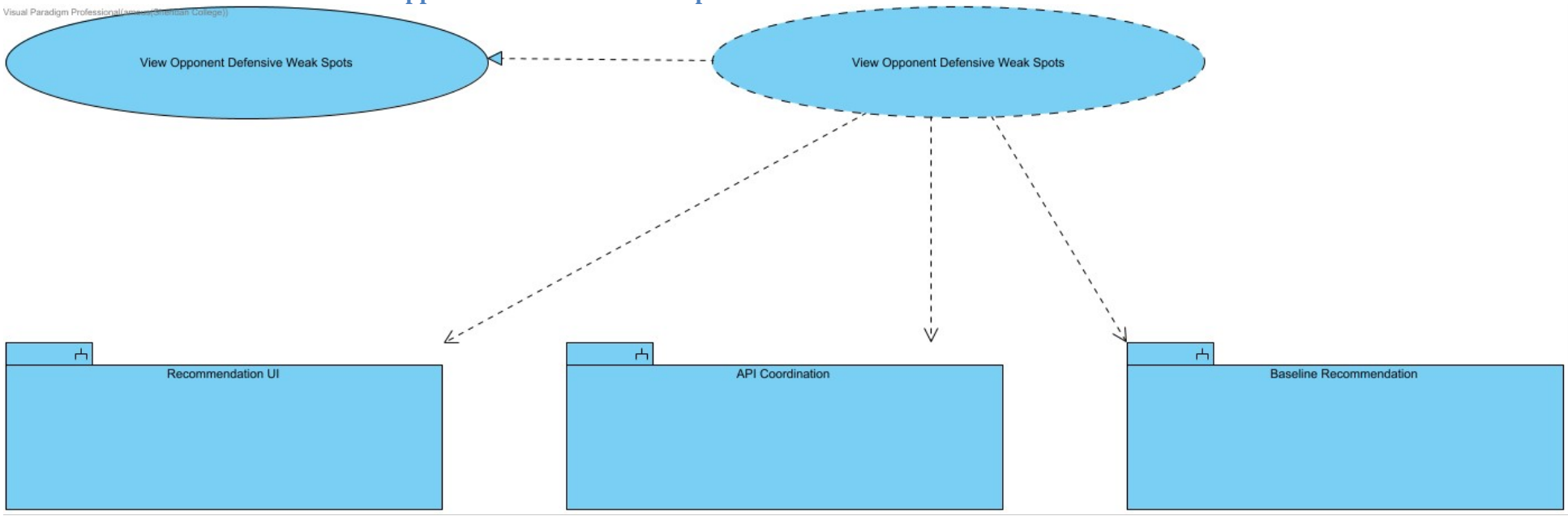


Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

4.2 View Opponent Defensive Weak Spots

Use case A308531b428a7bd1; SCRUM-696. The collaboration reuses the existing application and deployment layers. Numeric fields and alternate outcomes follow the SRS Flow of Events.

4.2.1 Collaboration Overview - View Opponent Defensive Weak Spots



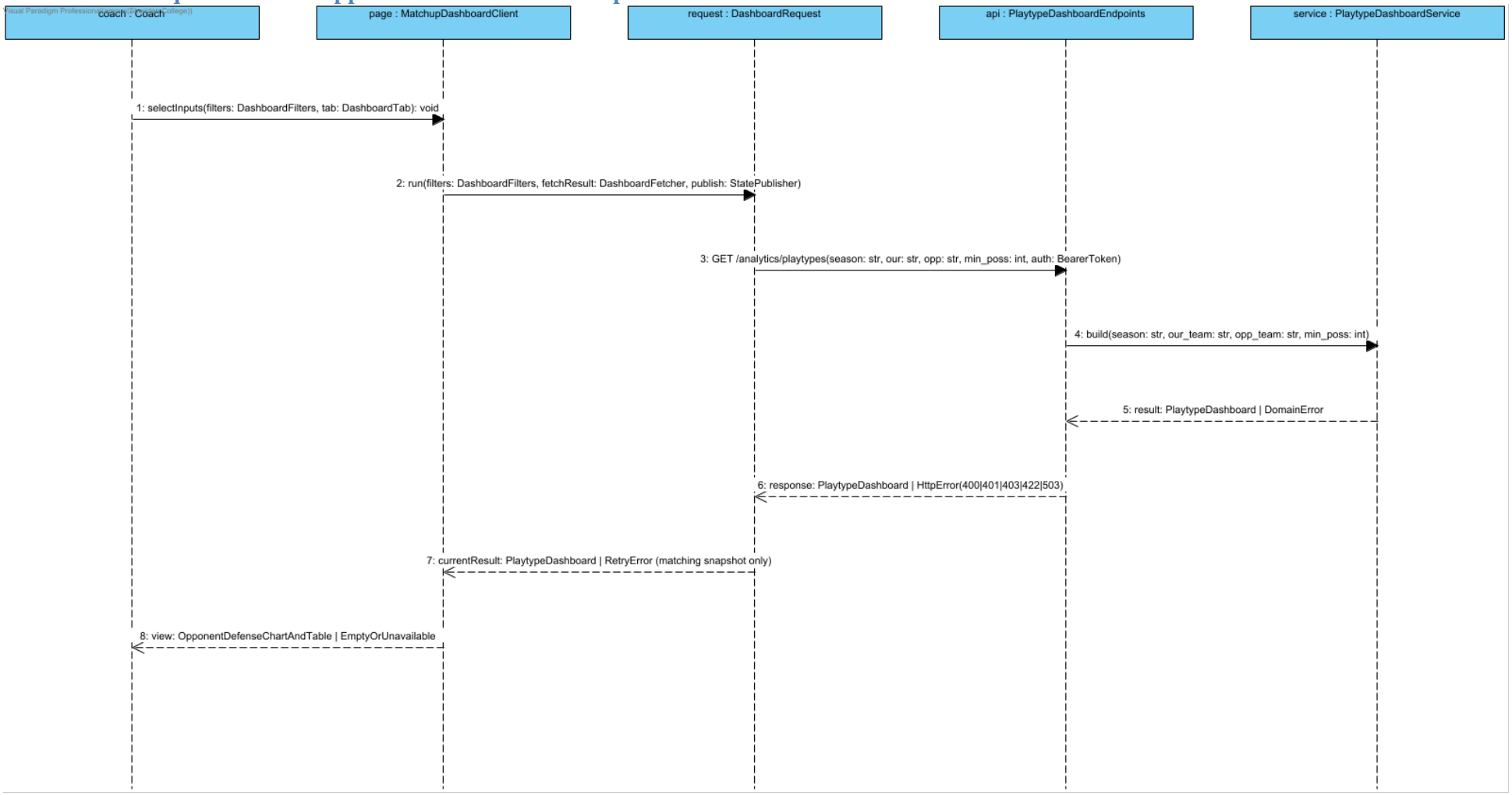
Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

4.2.2 Structural Design - View Opponent Defensive Weak Spots



Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

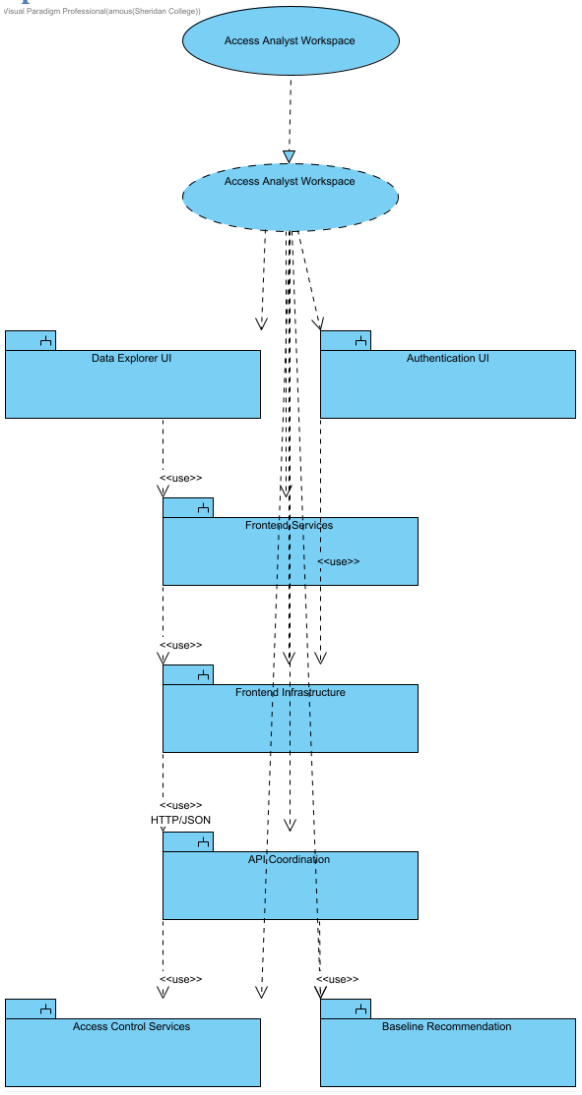
4.2.3 Detailed Sequence - View Opponent Defensive Weak Spots



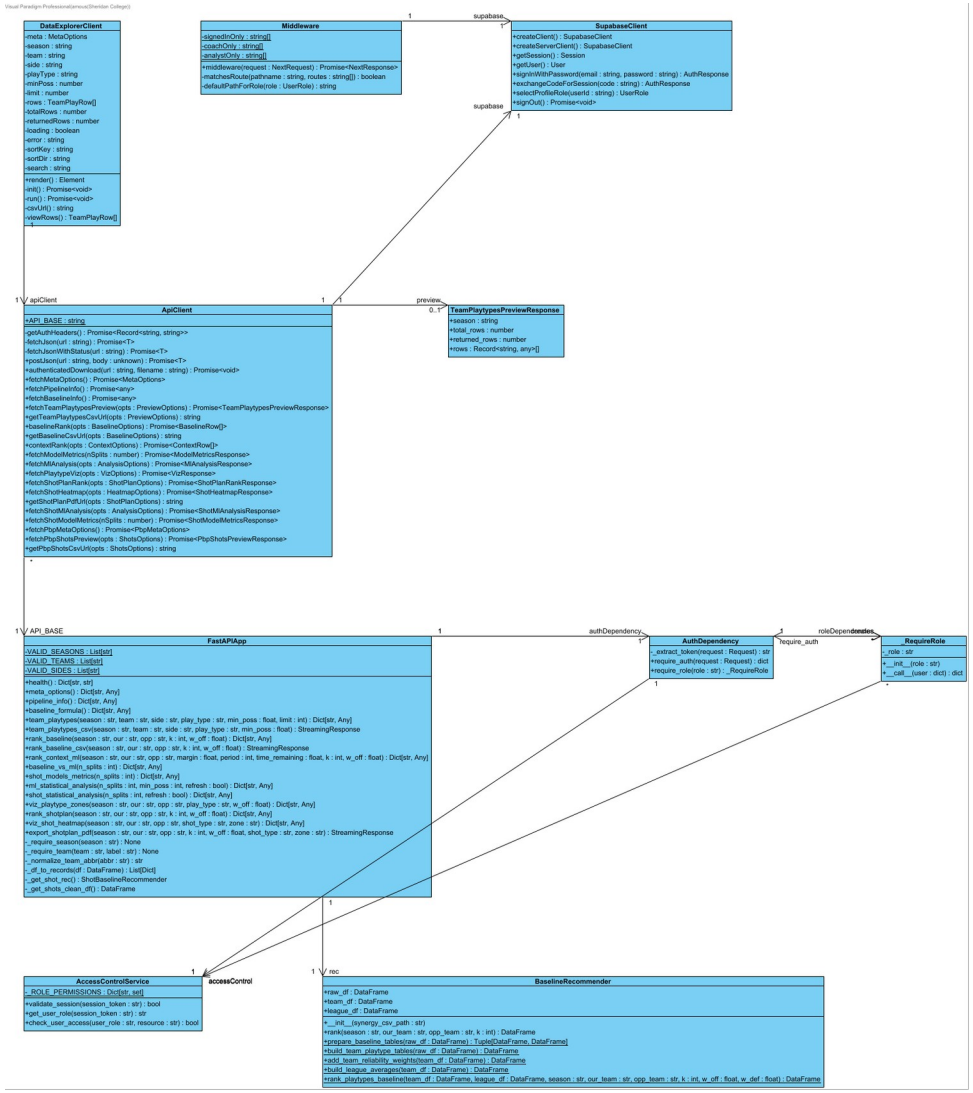
Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

4.3 Access Analyst Workspace

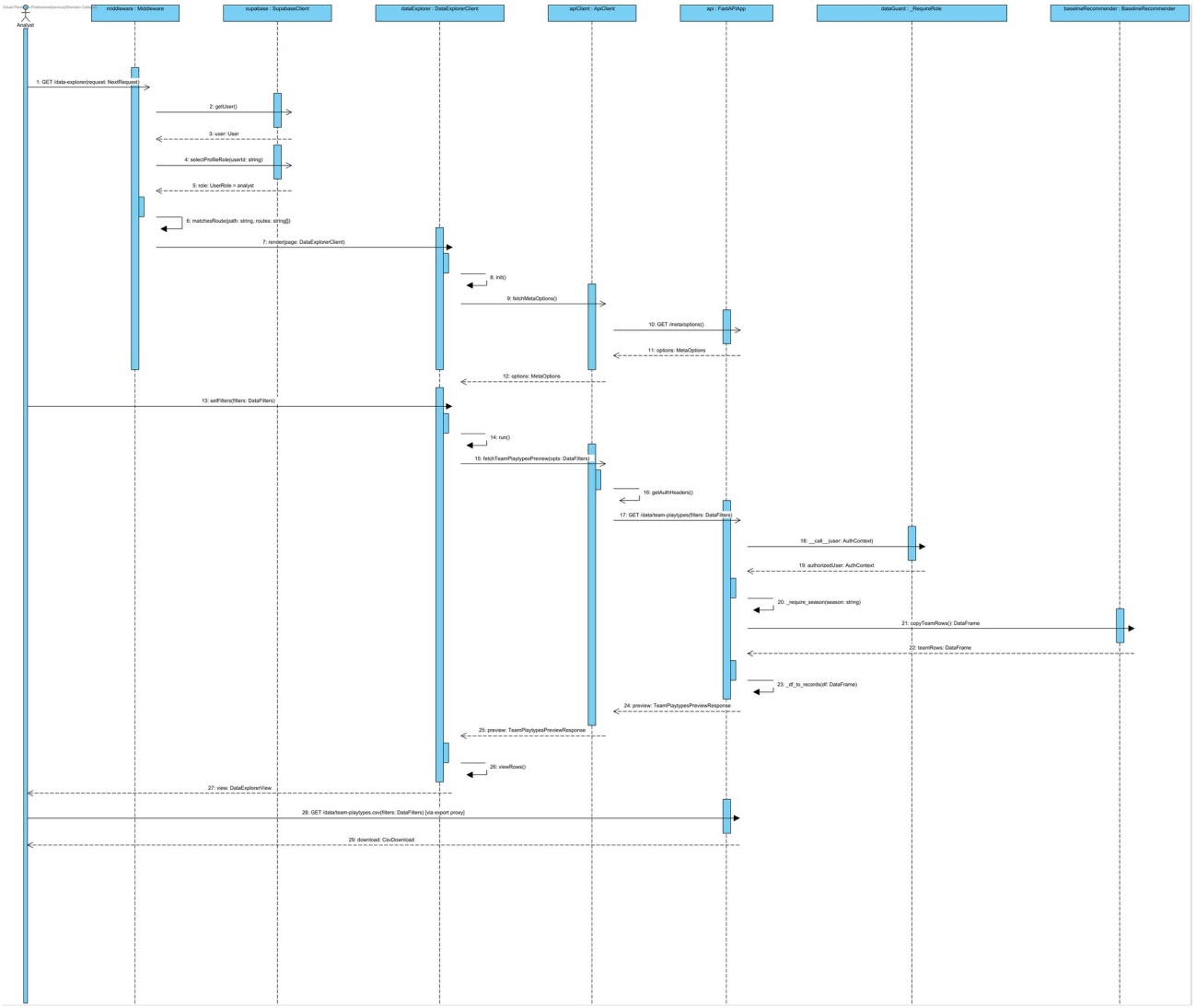
4.3.1 Collaboration Overview - Access Analyst Workspace



4.3.2 Structural Overview - Access Analyst Workspace



4.3.3 Behavioural Overview - Access Analyst Workspace

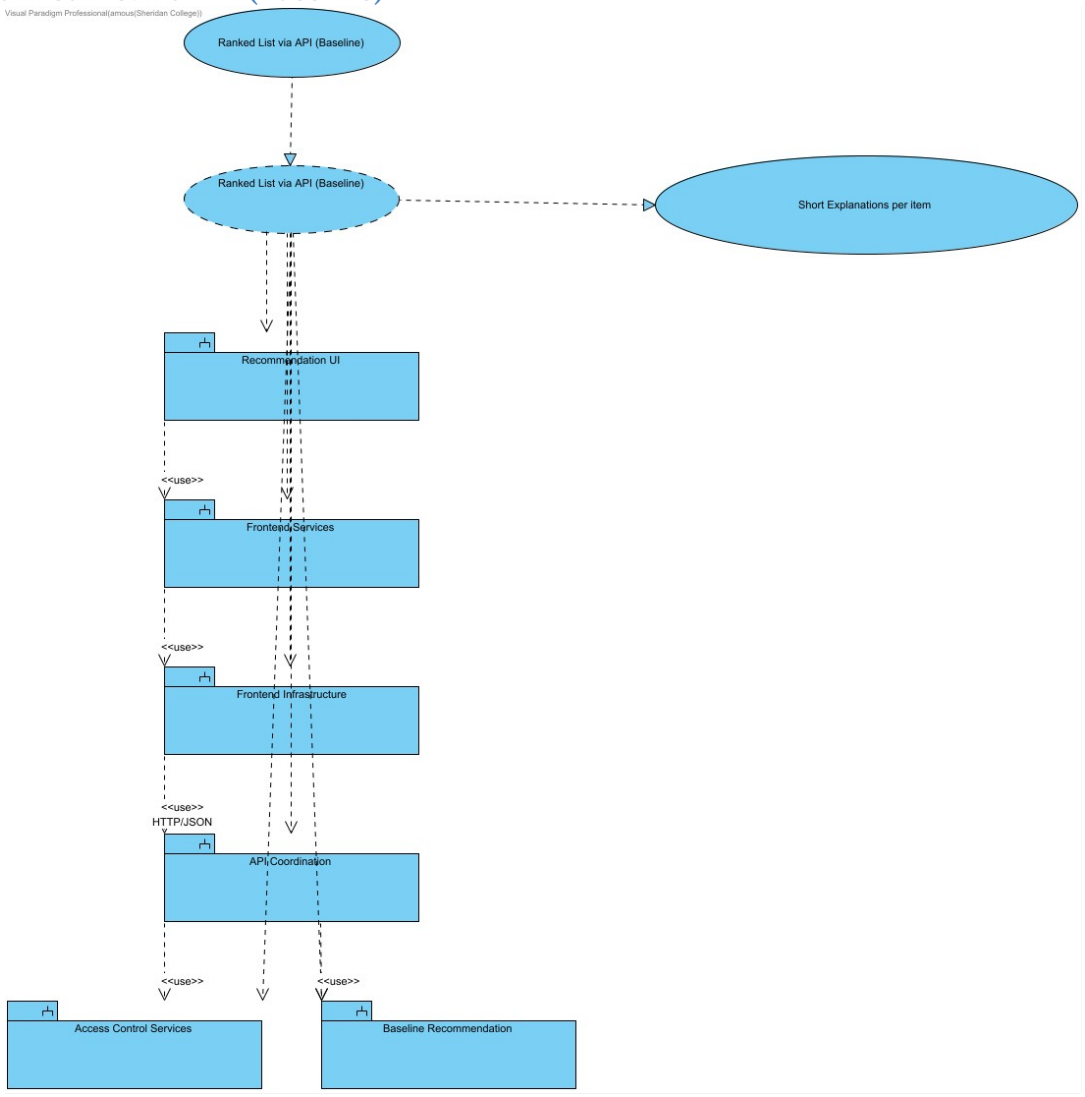


Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
DataFilters = {season: string, team?: string, side?: offense | defense, play_type?: string, min_poss: number >= 0, limit: integer 1..2000}; preview defaults limit=200. CSV uses the same filters except limit. TeamPlaytypesPreviewResponse = {season: string, total_rows: integer, returned_rows: integer, rows: TeamPlayRow[]}. TeamPlayRow contains the selected source columns, including PLAY_TYPE, POSS and PPP.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
CsvDownload = {status: 200, mediaType: text/csv, filename: string, body: UTF-8 bytes}. Credentials and concrete JWT values are intentionally absent. Promise completion returns the named payload; failures reject or return the labeled HTTP error and do not become successful exports.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

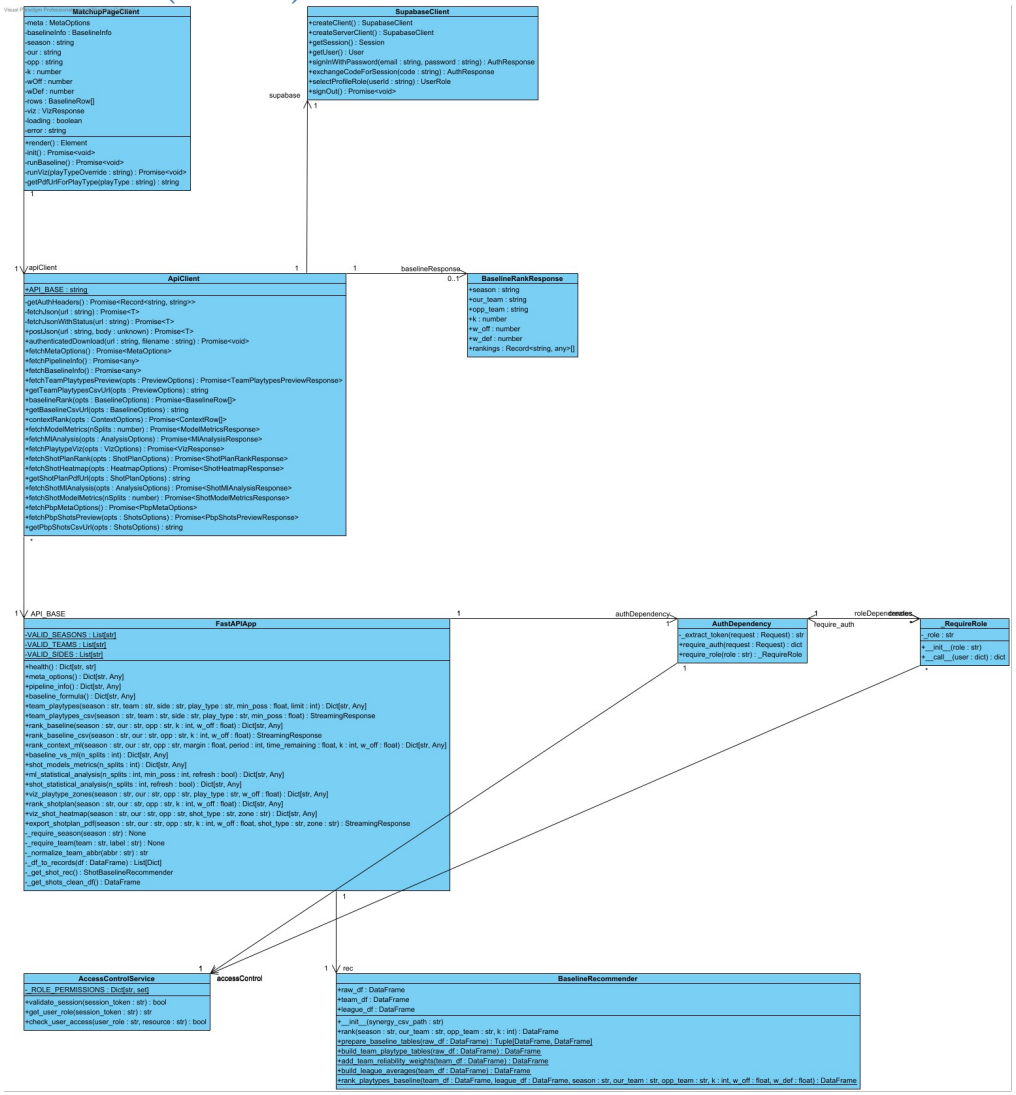
4.4 Ranked List via API (Baseline)

4.4.1 Collaboration Overview - Ranked List via API (Baseline)

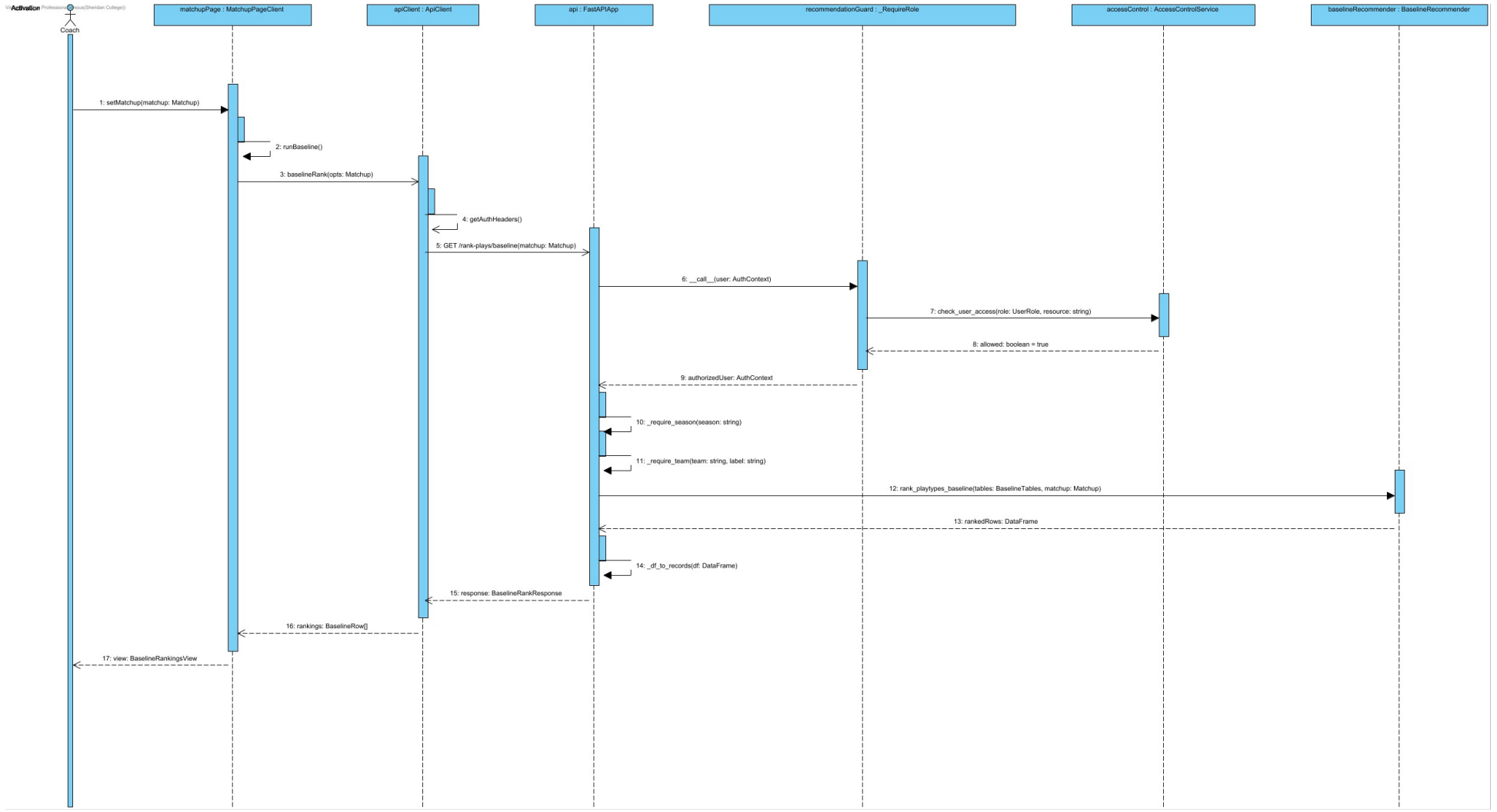


SCRUM-546: this collaboration also supplies short explanations for its own current ranking. Baseline explains season aggregates and unused score/period/clock; context identifies the conditions actually applied. No causal or future-accuracy guarantee is added.

4.4.2 Structural Overview - Ranked List via API (Baseline)



4.4.3 Behavioural Overview - Ranked List via API (Baseline)

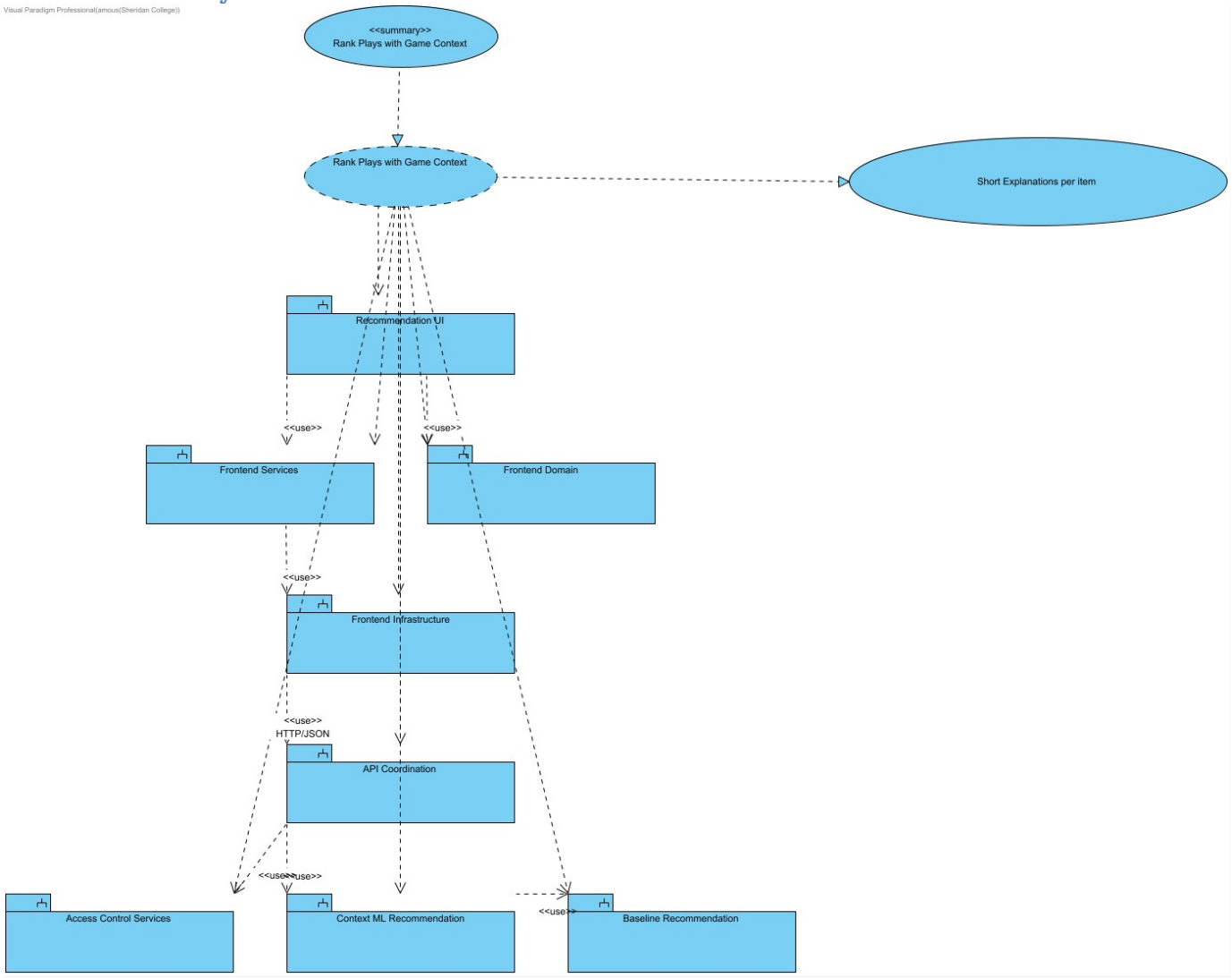


Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

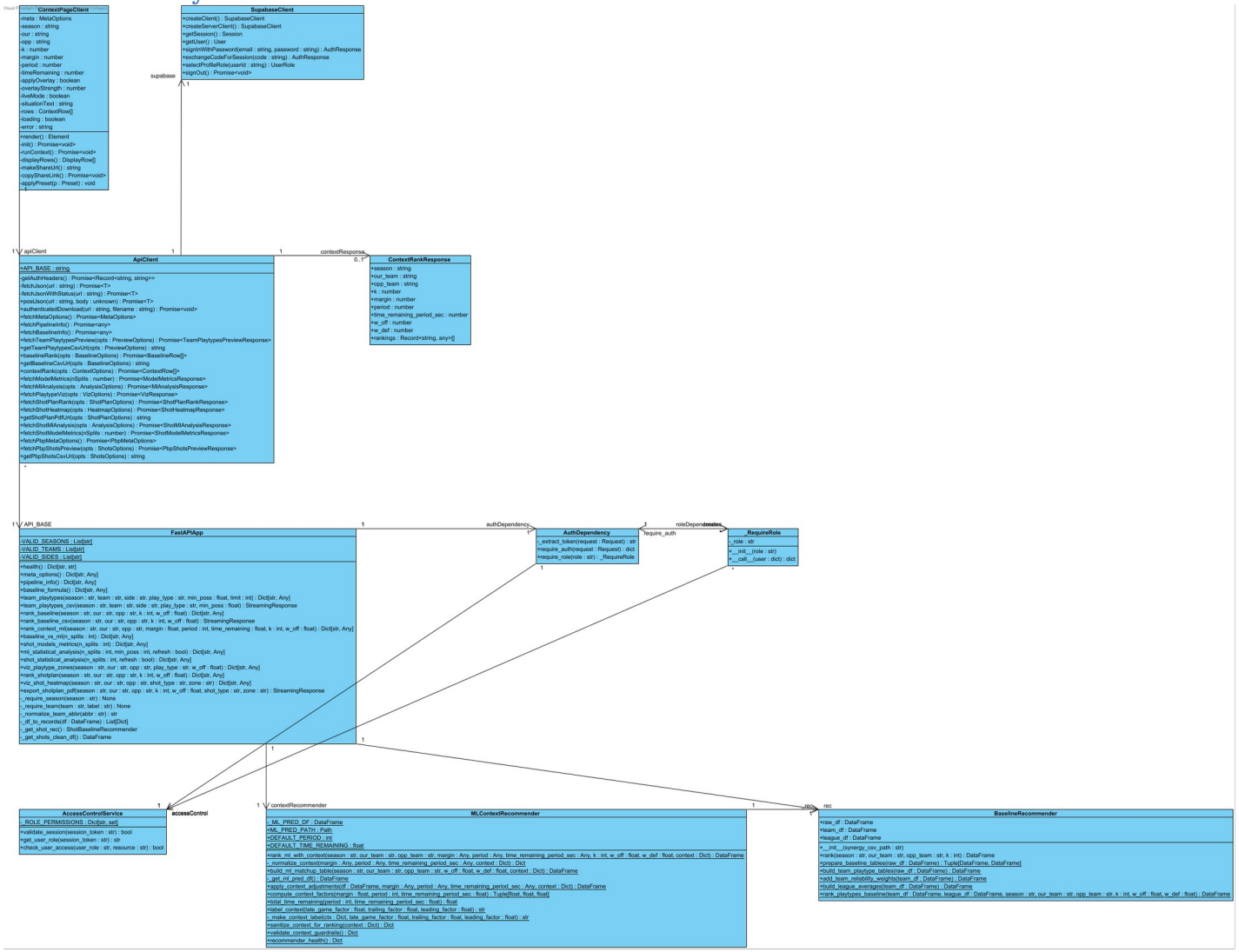
4.5 Rank Plays with Game Context

4.5.1 Collaboration Overview - Rank Plays with Game Context

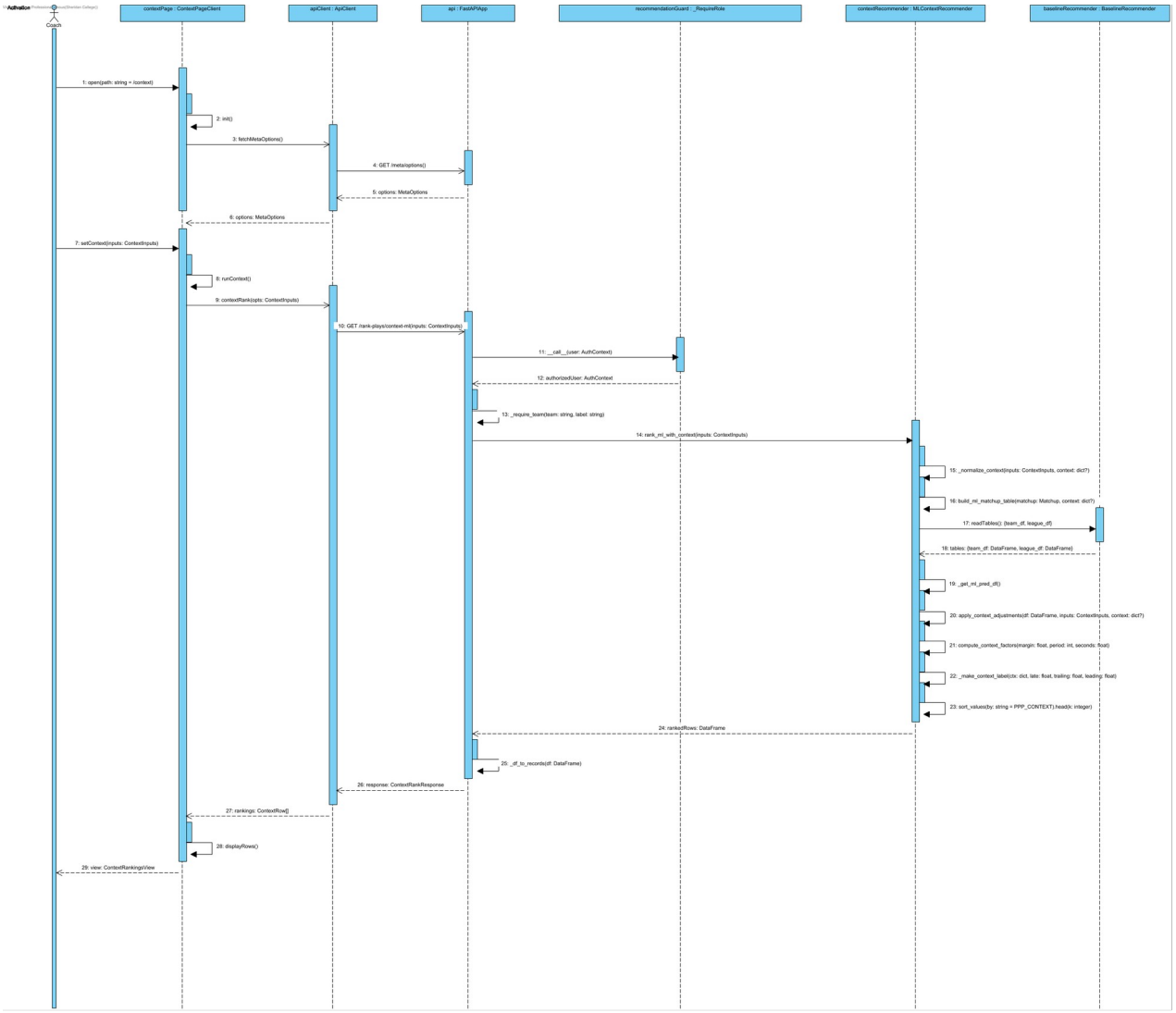


SCRUM-546: this collaboration also supplies short explanations for its own current ranking. Baseline explains season aggregates and unused score/period/clock; context identifies the conditions actually applied. No causal or future-accuracy guarantee is added.

4.5.2 Structural Overview - Rank Plays with Game Context



4.5.3 Behavioural Overview - Rank Plays with Game Context

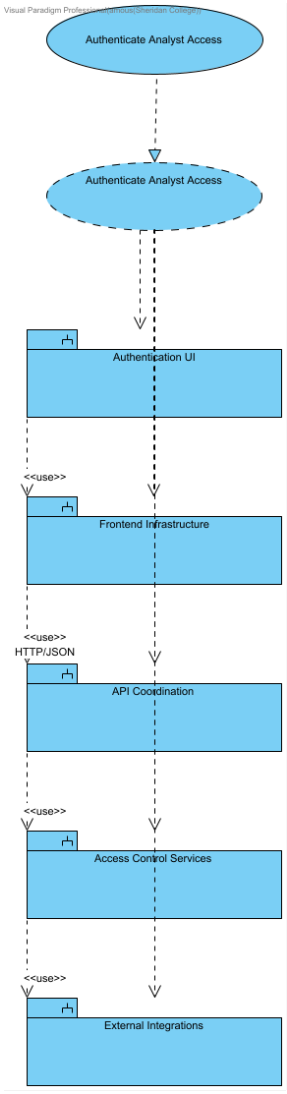


Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

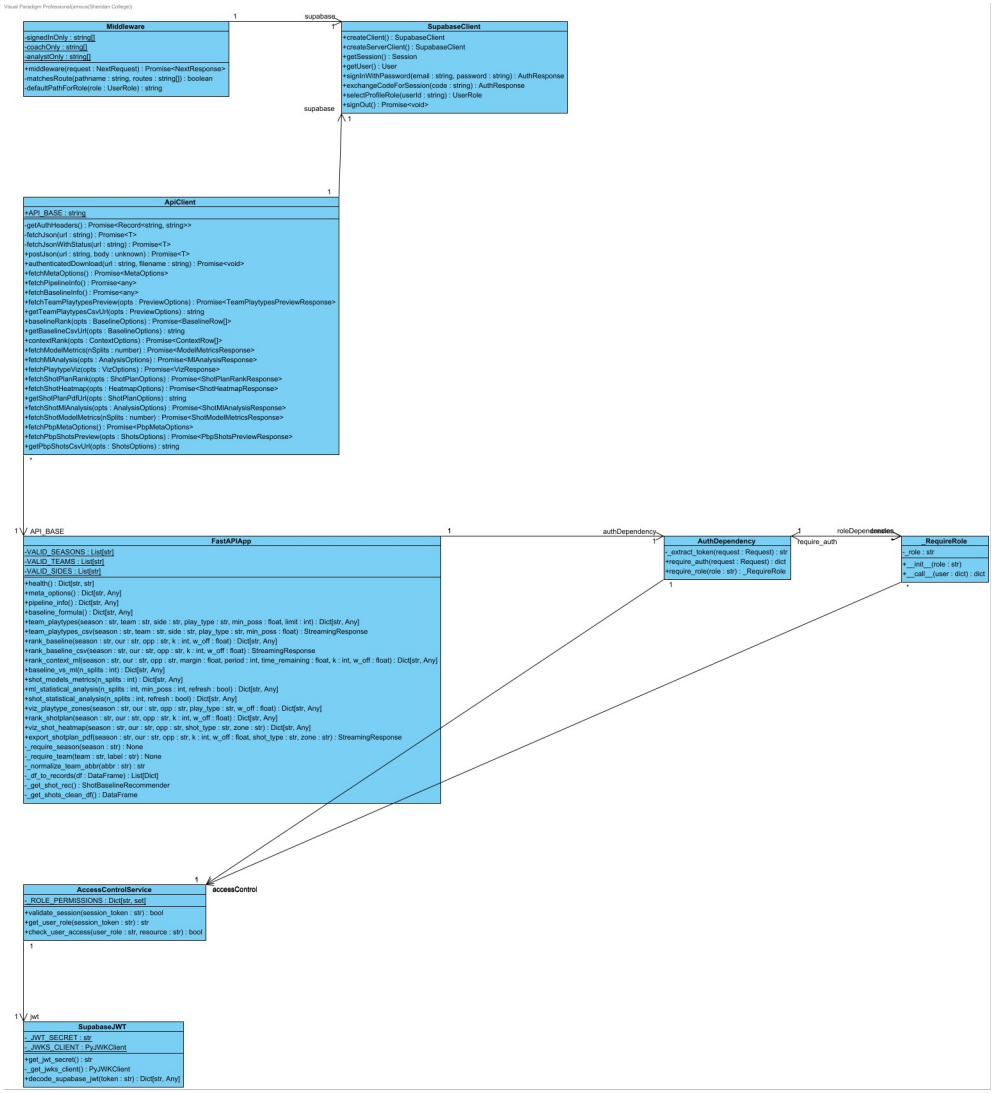
Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

4.6 Authenticate Analyst Access

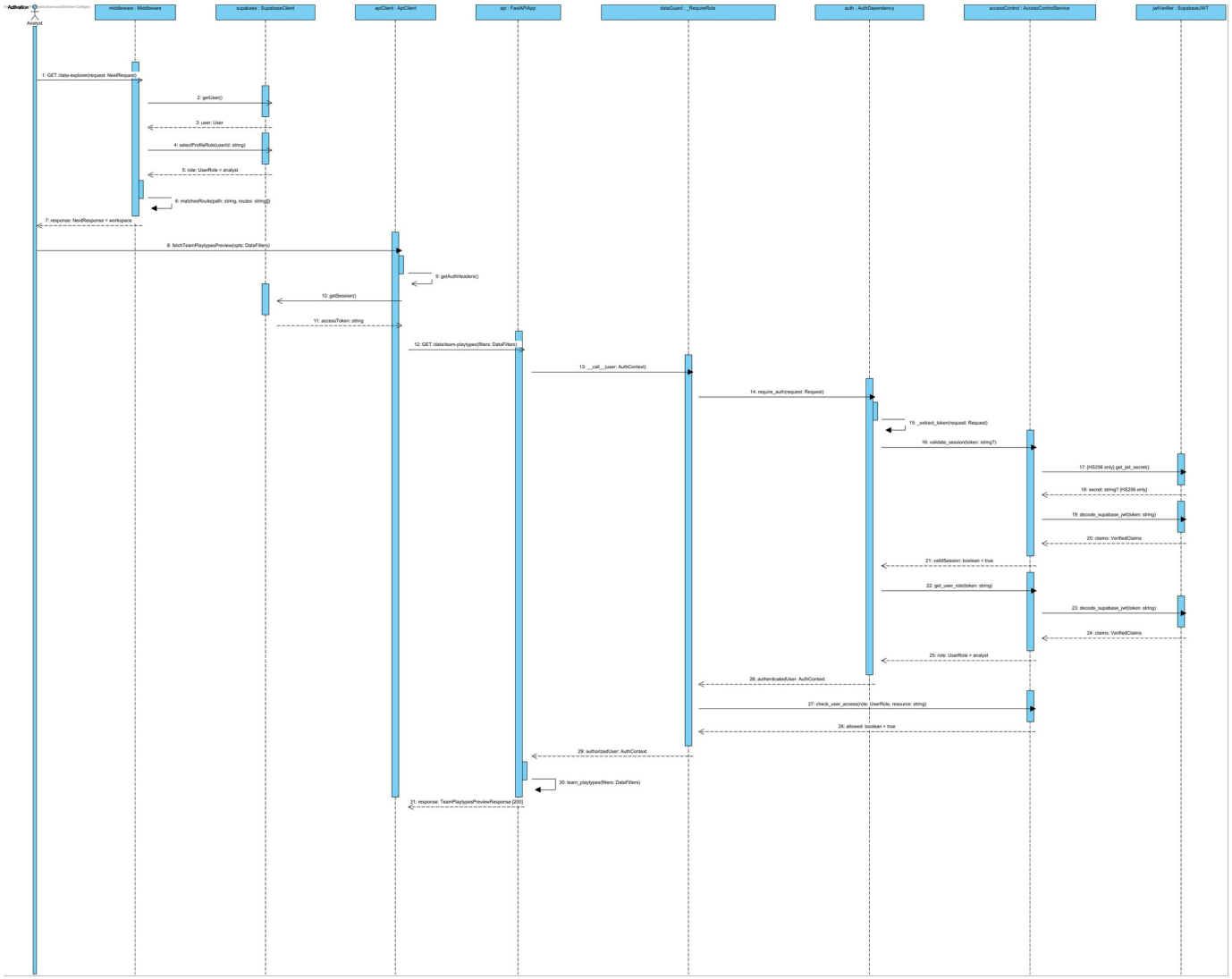
4.6.1 Collaboration Overview - Authenticate Analyst Access



4.6.2 Structural Overview - Authenticate Analyst Access



4.6.3 Behavioural Overview - Authenticate Analyst Access

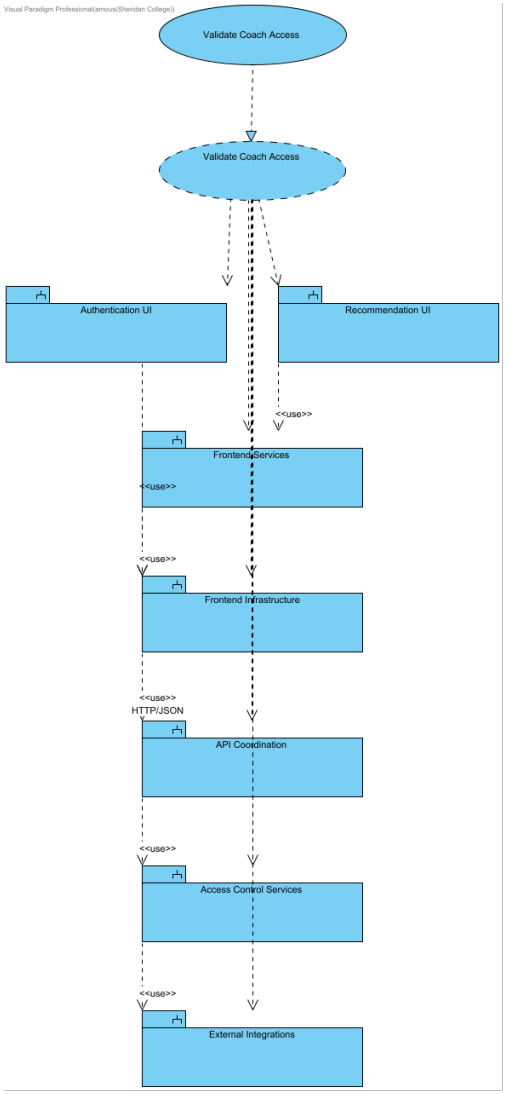


Production authorization validates ES256/RS256 JWT signatures against Supabase JWKS plus issuer, audience and expiry; verified subject identifies the user. The current protected public.profiles.role authorizes resources, never user_metadata.role. Unassigned accounts remain pending. Analyst evidence endpoints require analyst access; coach recommendations and the matchup dashboard require coach access. Shared shot views accept either approved role. Invalid tokens return 401 and forbidden roles return 403. Current explicit protected-profile authorization exchange is shown in Detailed Sequence - Analyst Top-K Uplift (475).Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call

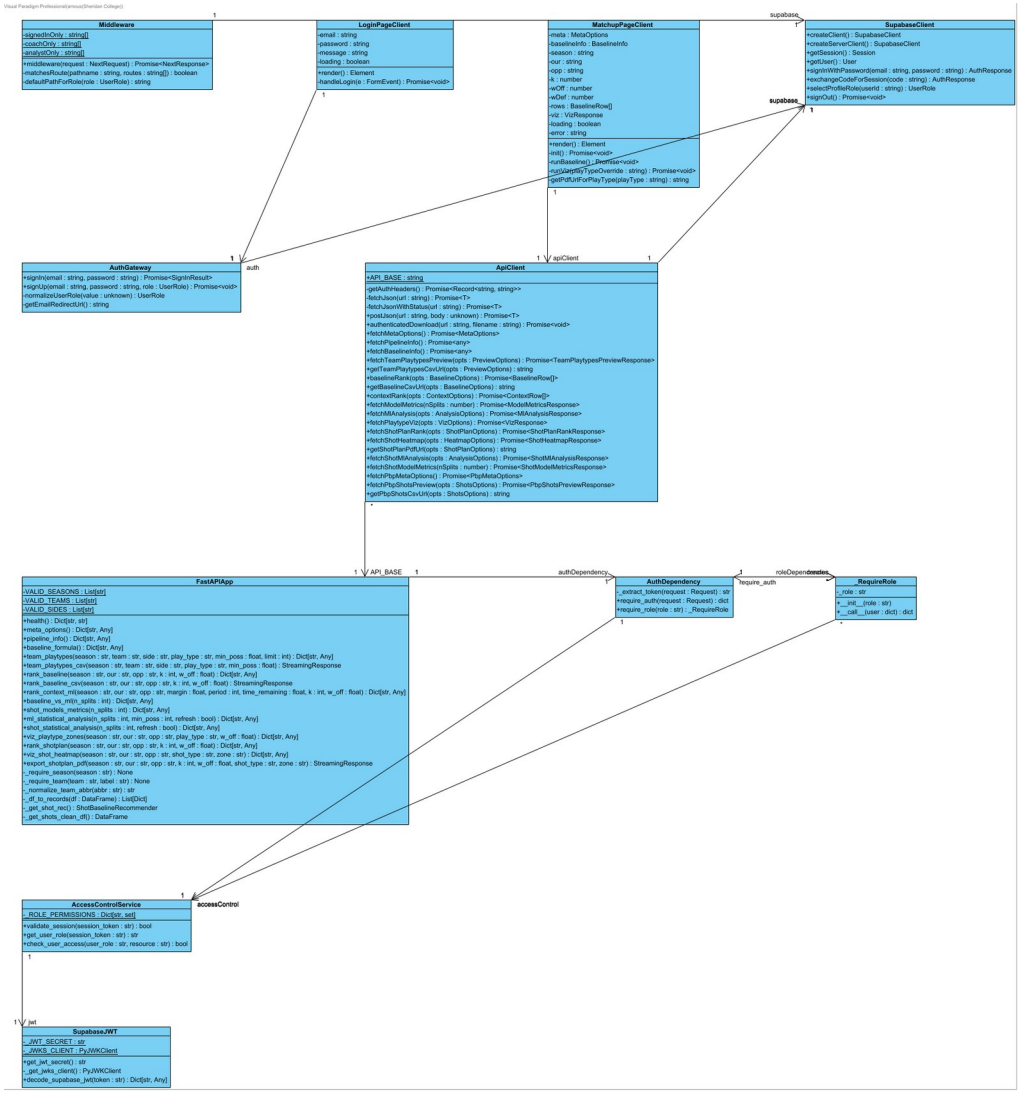
arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

4.7 Validate Coach Access

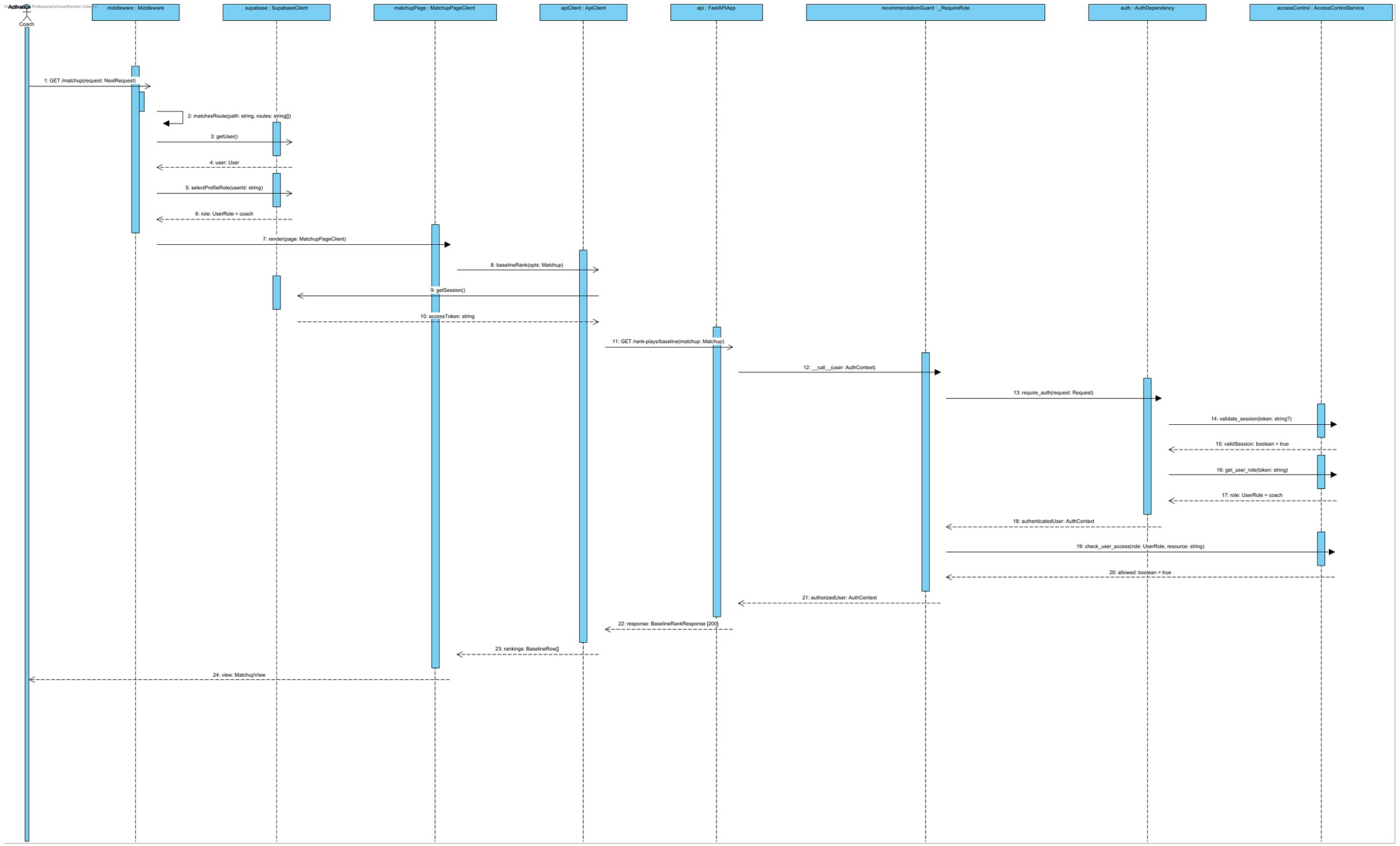
4.7.1 Collaboration Overview - Validate Coach Access



4.7.2 Structural Overview - Validate Coach Access



4.7.3 Behavioural Overview - Validate Coach Access



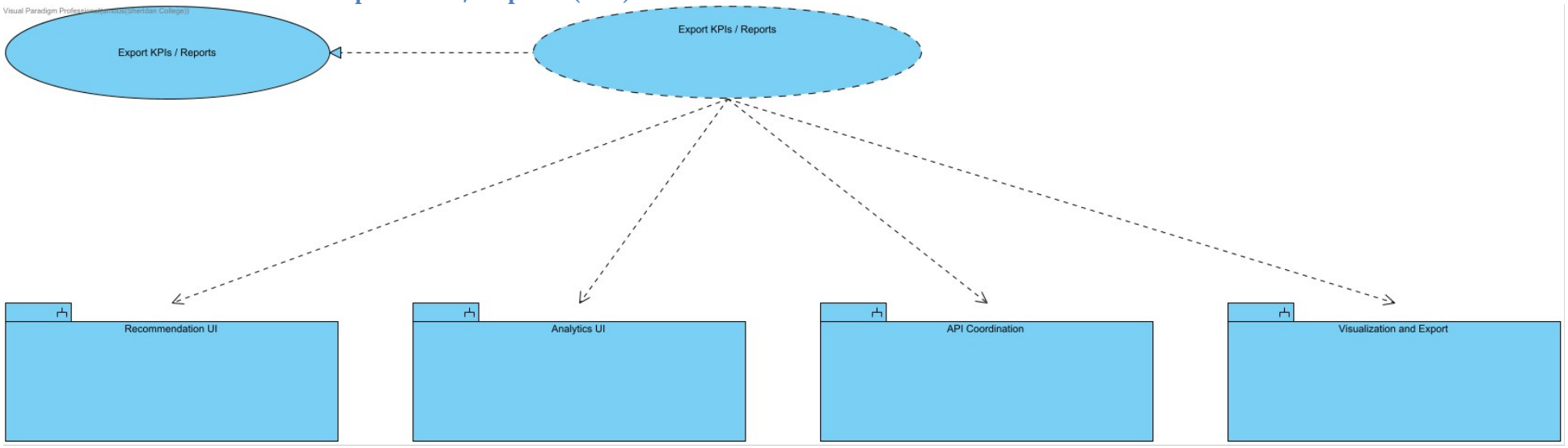
Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

4.8 Export KPIs / Reports

The structural design for this interaction is the Shared Export Controls class diagram and its detailed sequence, shown under API Coordination in the Detailed Design section.

4.8.1 Collaboration Overview - Export KPIs / Reports (486)

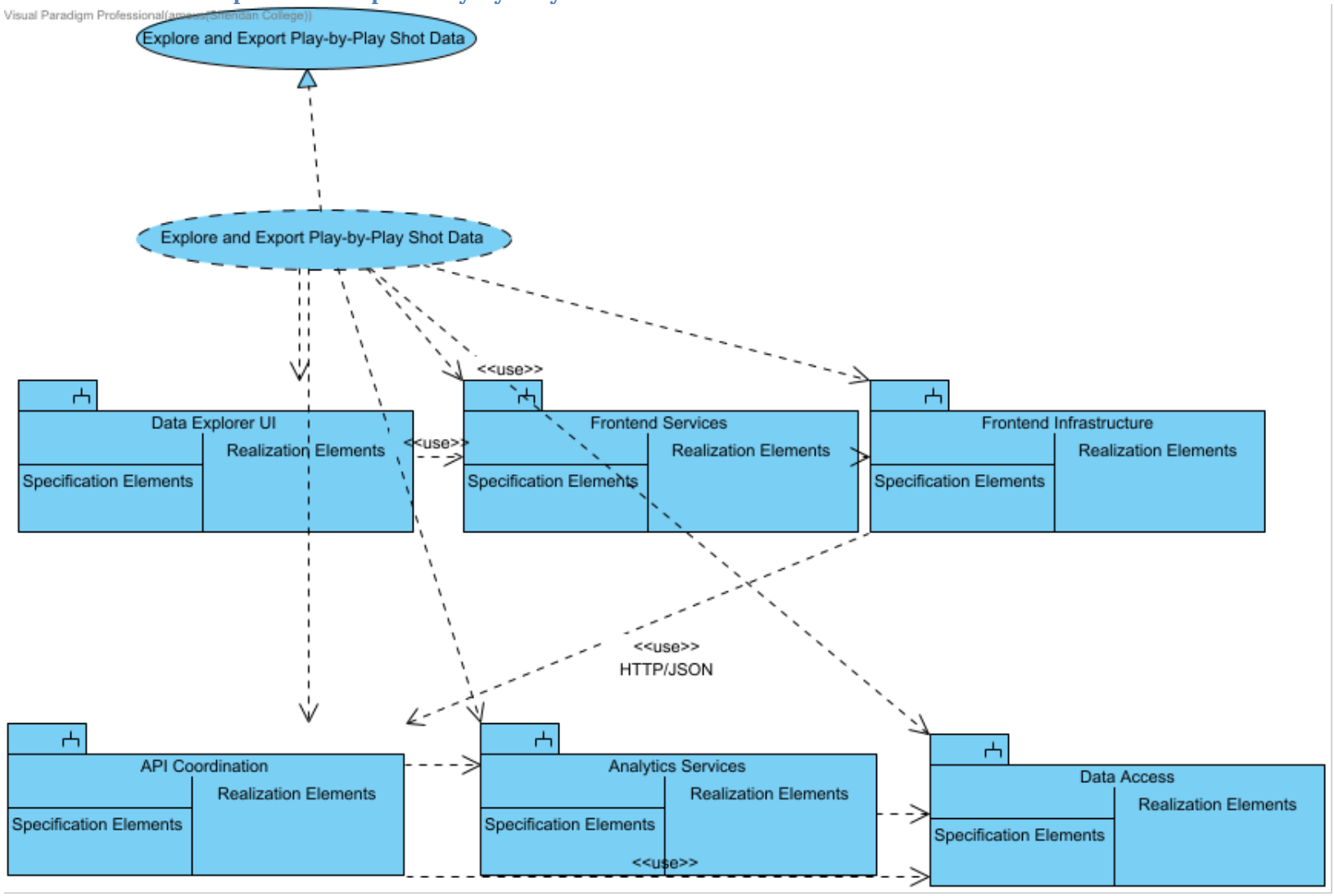


SCRUM-486. Export KPIs / Reports. This collaboration realizes the linked use case and owns its overview, structural design and detailed sequence. Modules remain in the Design Model. Implements the analyst Export KPIs / Reports use case. The same shared control is also reused by coach Matchup and Context; their existing use cases and collaboration are unchanged. Only displayed results are exported.

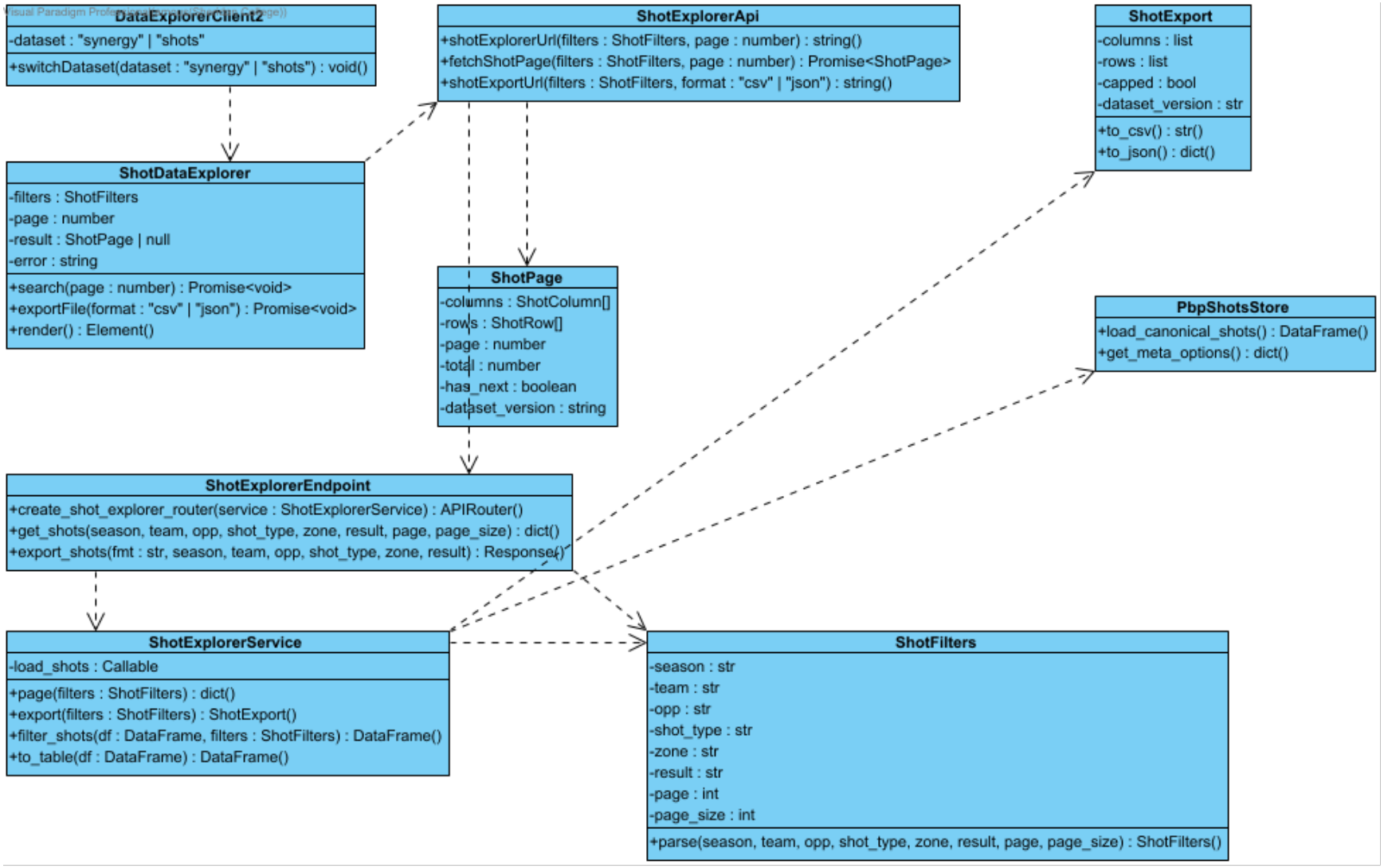
4.9 Explore and Export Play-by-Play Shot Data

The analyst switches the Data Explorer to the play-by-play shot dataset, filters the shots and exports the same rows to CSV or JSON. ShotExplorerService validates the filters and builds both the page and the export from one result, so the table and the files can never disagree. A missing season or team returns a 400 with a plain message, and a missing shot file returns a 503 instead of an empty table.

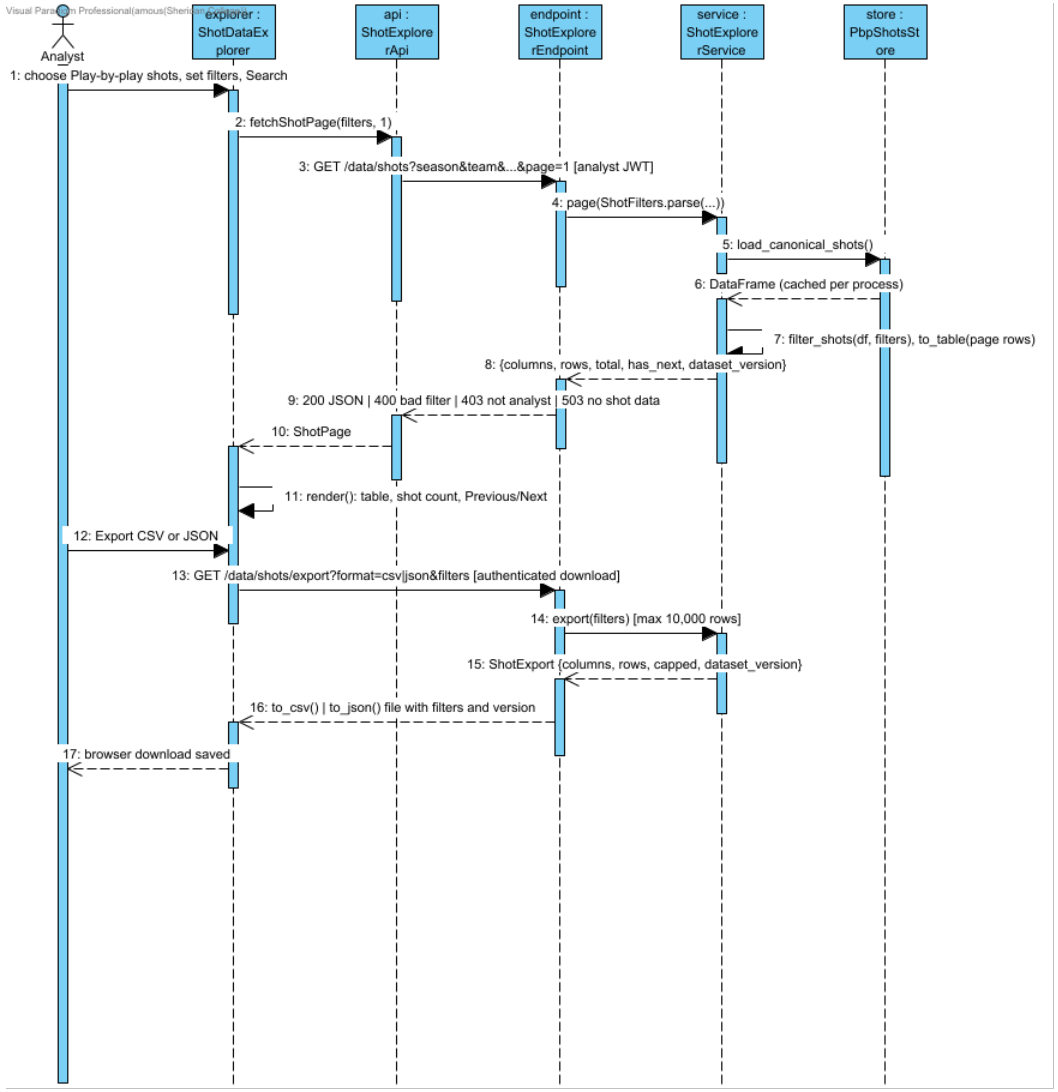
4.9.1 Collaboration Overview - Explore and Export Play-by-Play Shot Data



4.9.2 Structural Overview - Explore and Export Play-by-Play Shot Data



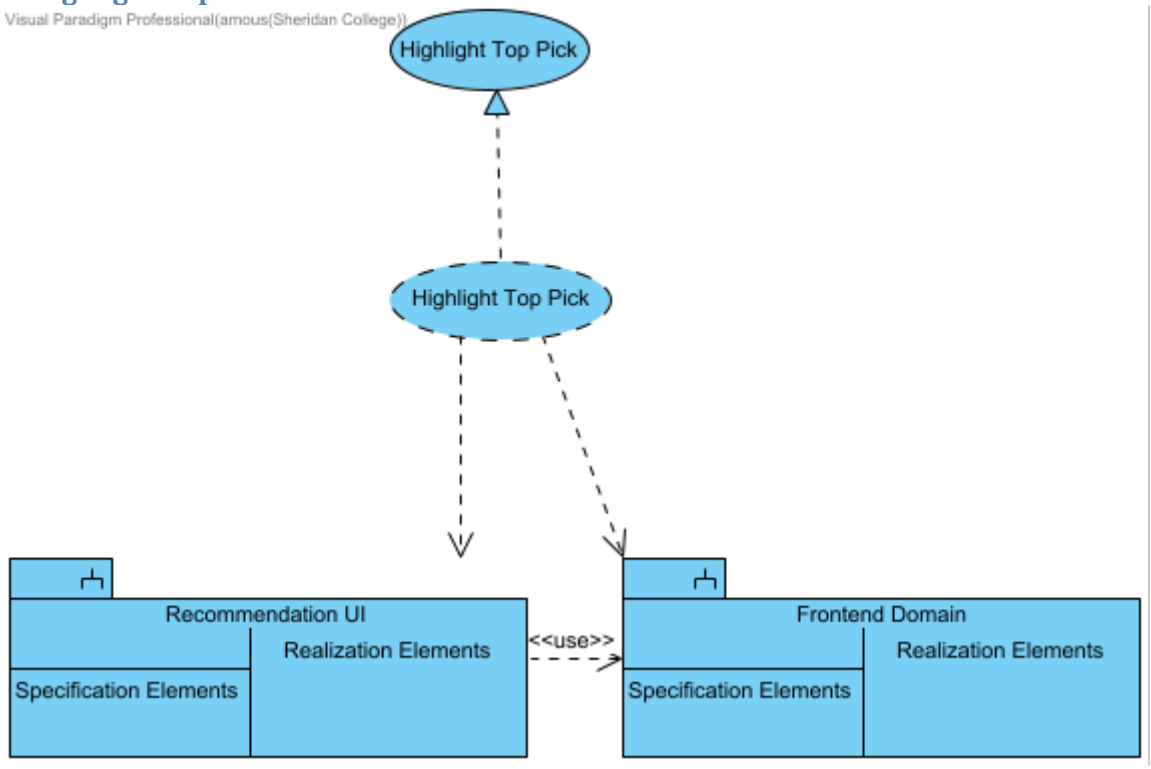
4.9.3 Behavioural Overview - Explore and Export Play-by-Play Shot Data



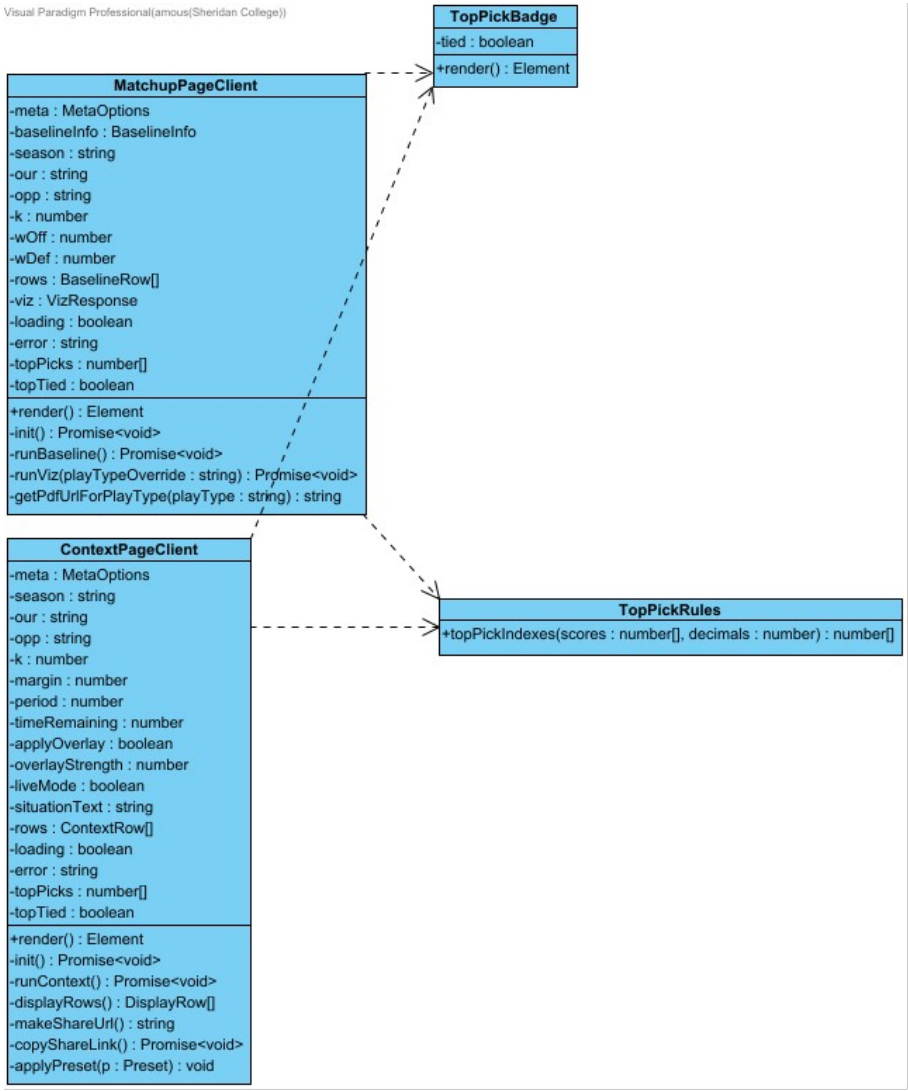
4.10 Highlight Top Pick

The Matchup and Context pages ask TopPickRules for the index of the top play and of any play tied with it on the score rounded to three decimals, then render a TopPickBadge on those rows. No API change is needed, because the highlight is worked out from the scores the API already returns.

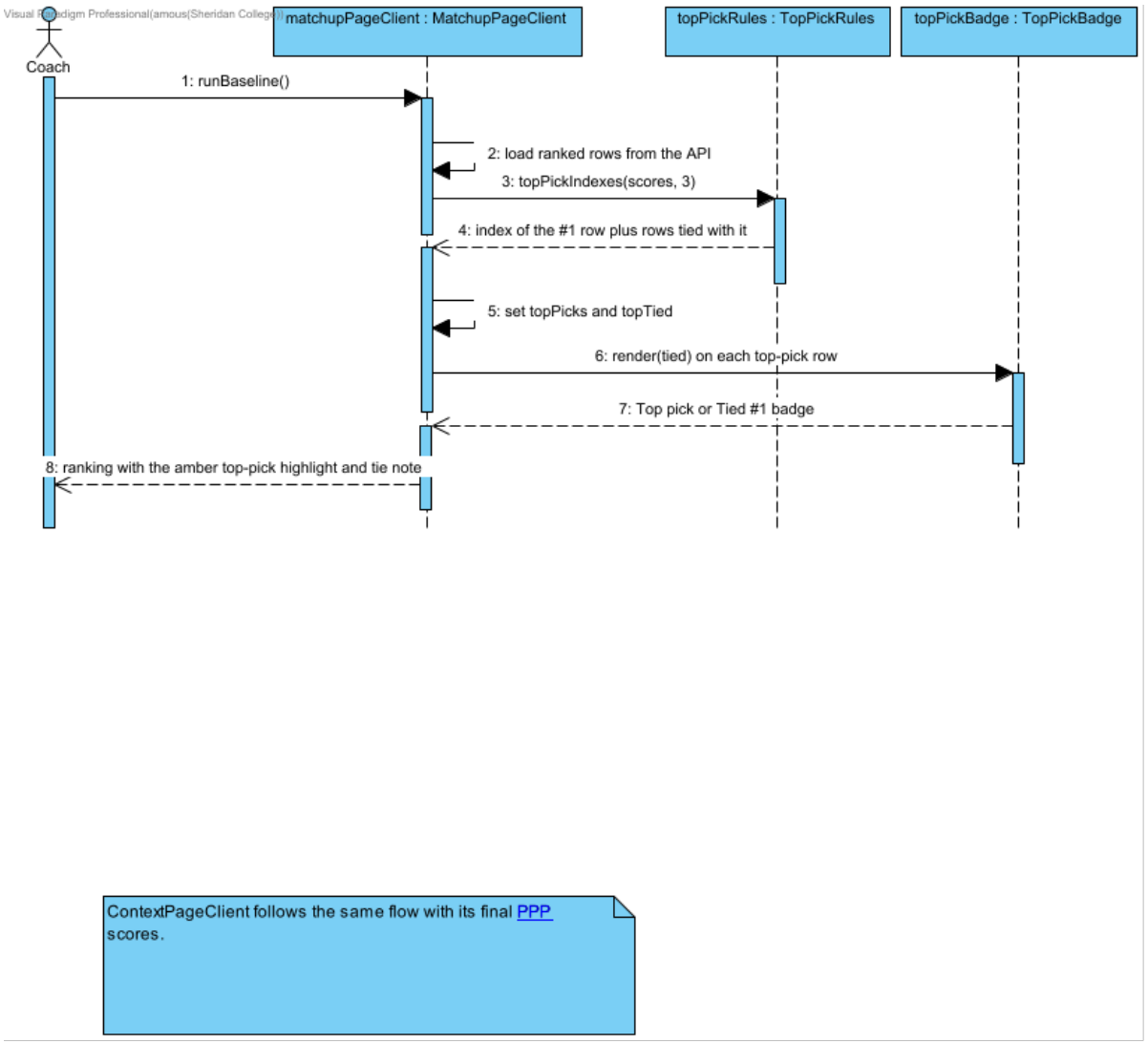
4.10.1 Collaboration Overview - Highlight Top Pick



4.10.2 Structural Overview - Highlight Top Pick



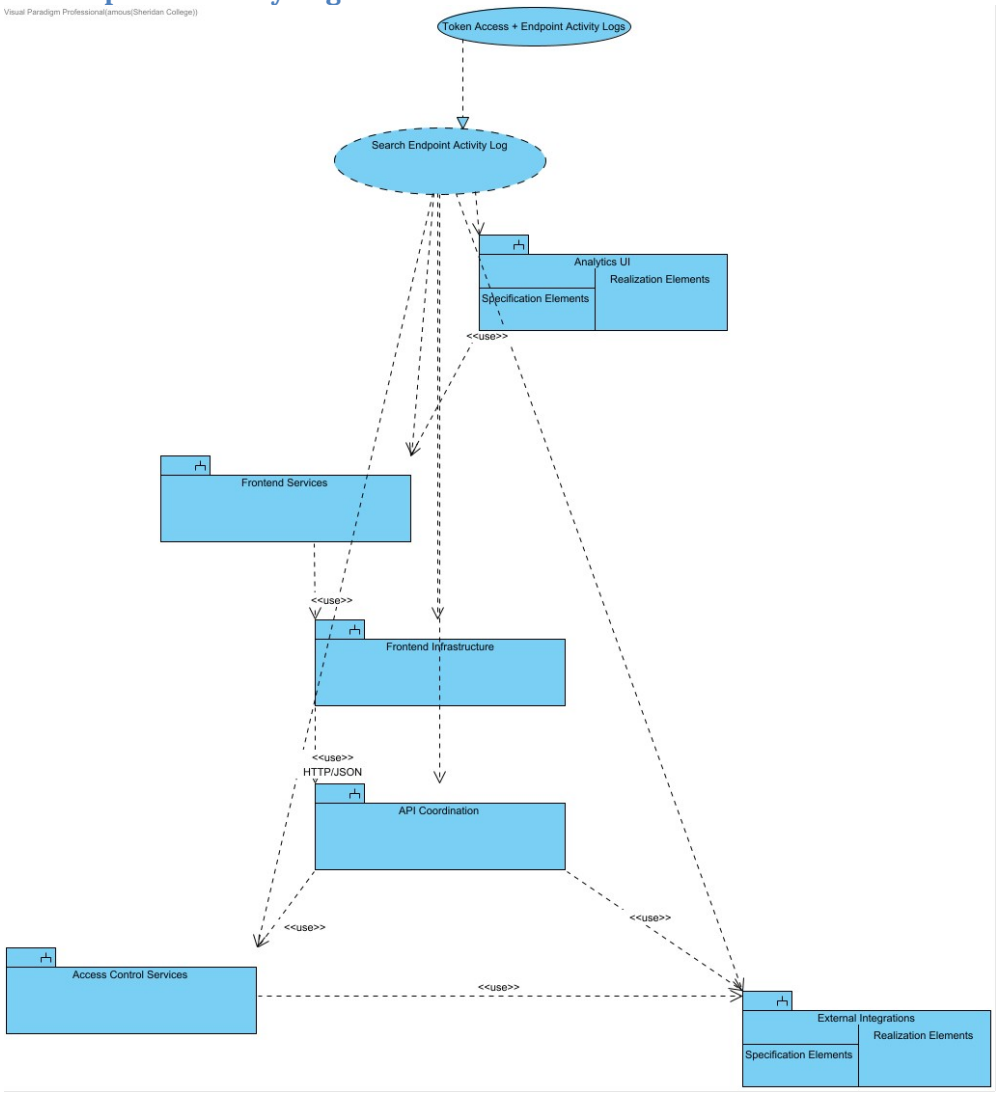
4.10.3 Behavioural Overview - Highlight Top Pick



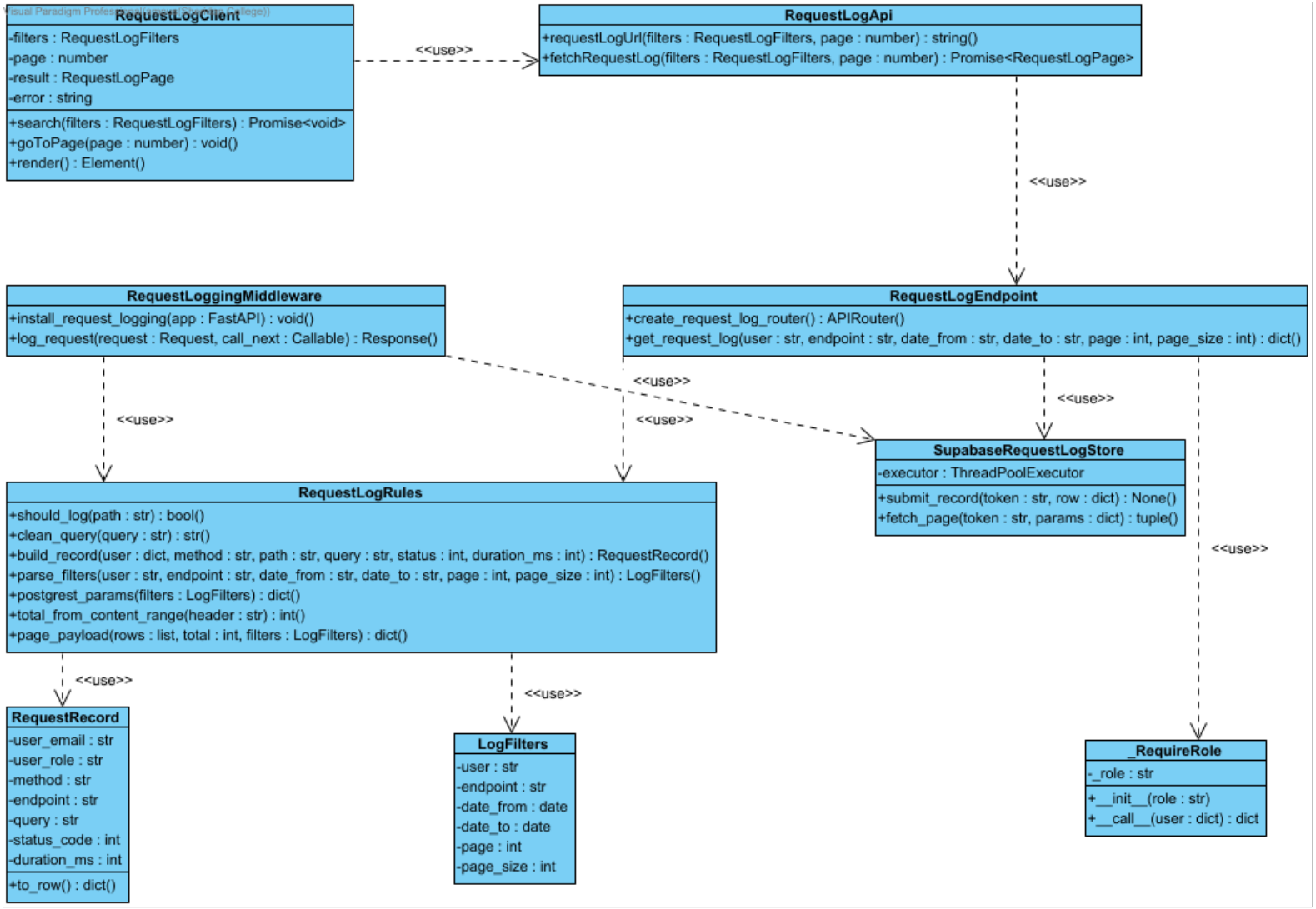
4.11 Search Endpoint Activity Log

RequestLoggingMiddleware builds a log record for every authenticated call, drops credential-like query values through RequestLogRules and hands the record to SupabaseRequestLogStore in the background. The store retries once on server errors or rate limits and flushes pending writes when the API shuts down, so a restart does not lose log rows. Calls without a verified user are rejected with a 401 and are not logged.

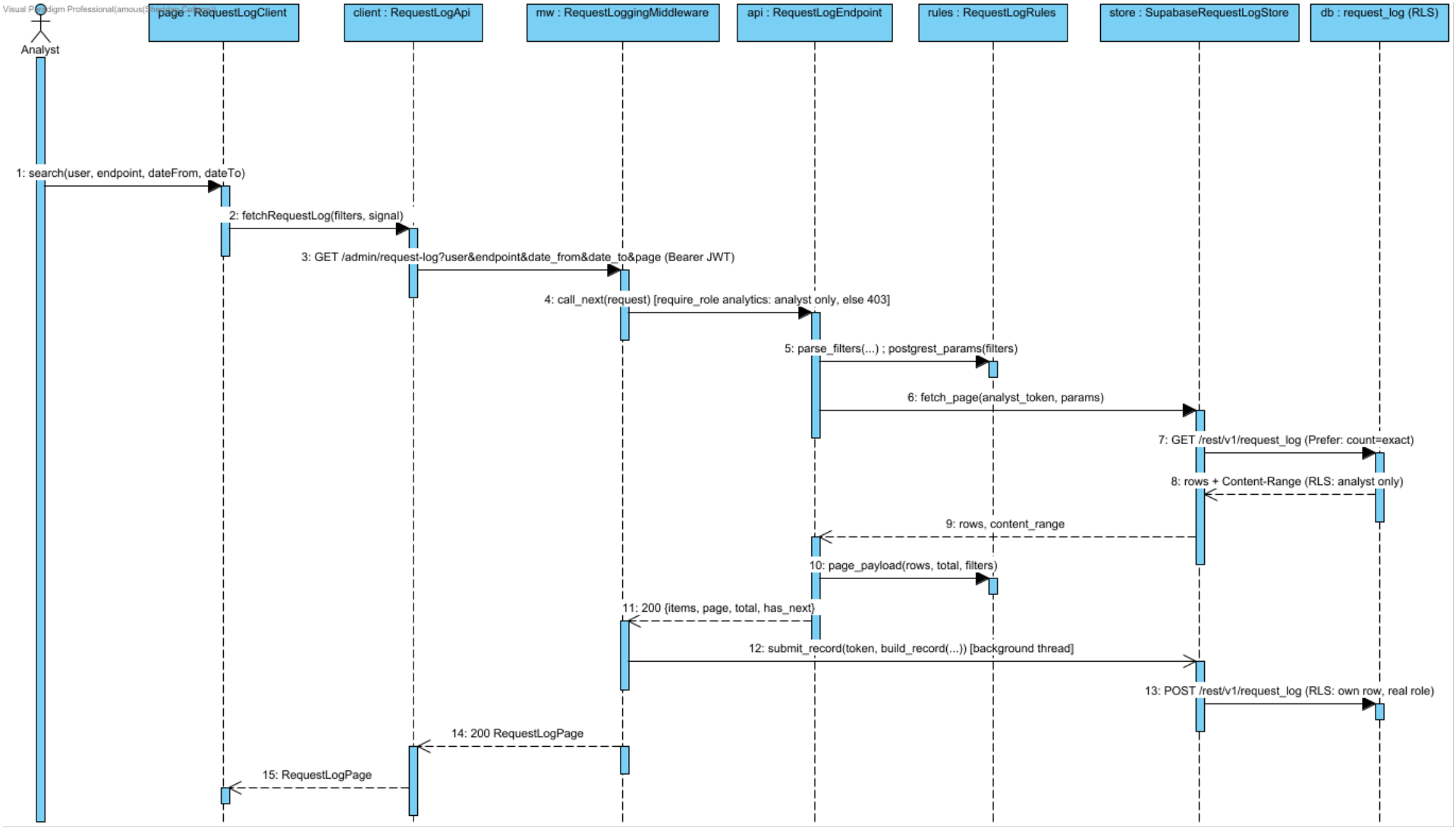
4.11.1 Collaboration Overview - Search Endpoint Activity Log



4.11.2 Structural Overview - Search Endpoint Activity Log



4.11.3 Behavioural Overview - Record and Search Request Log



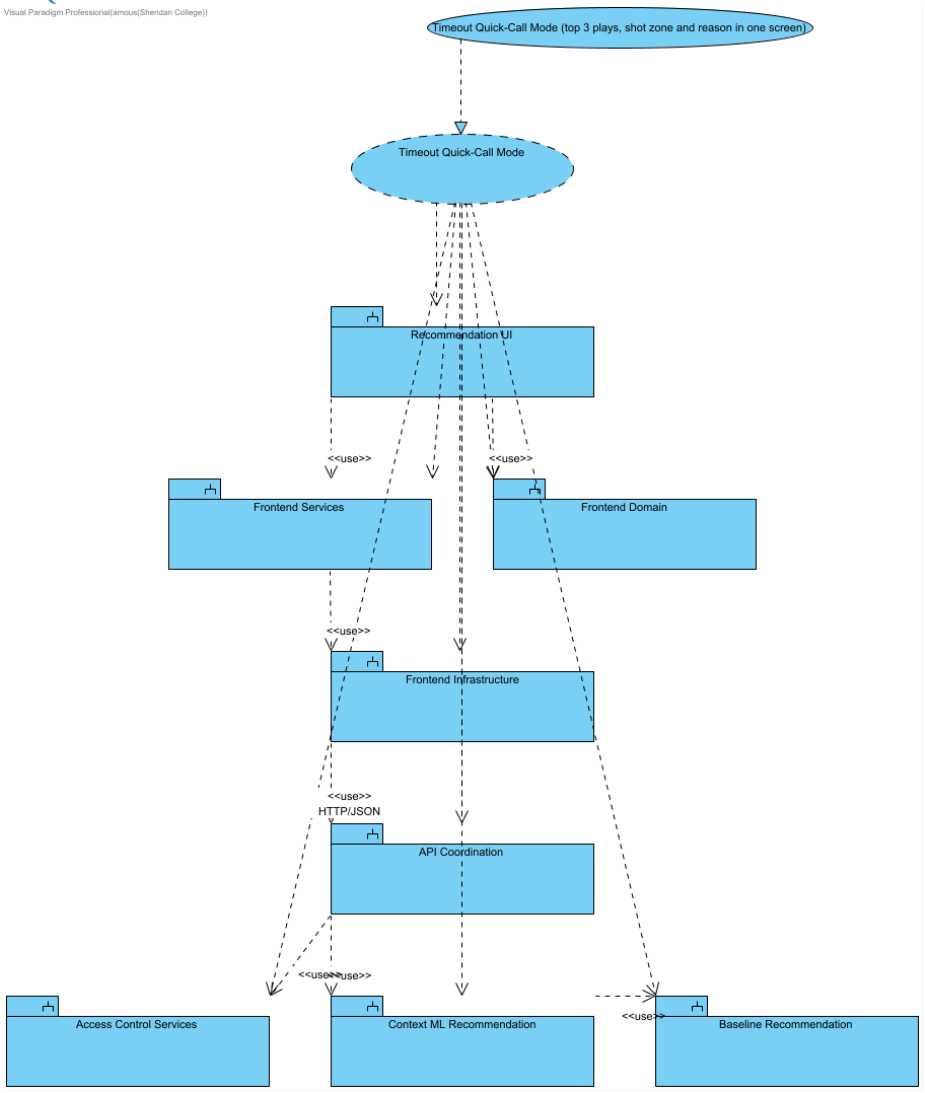
Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

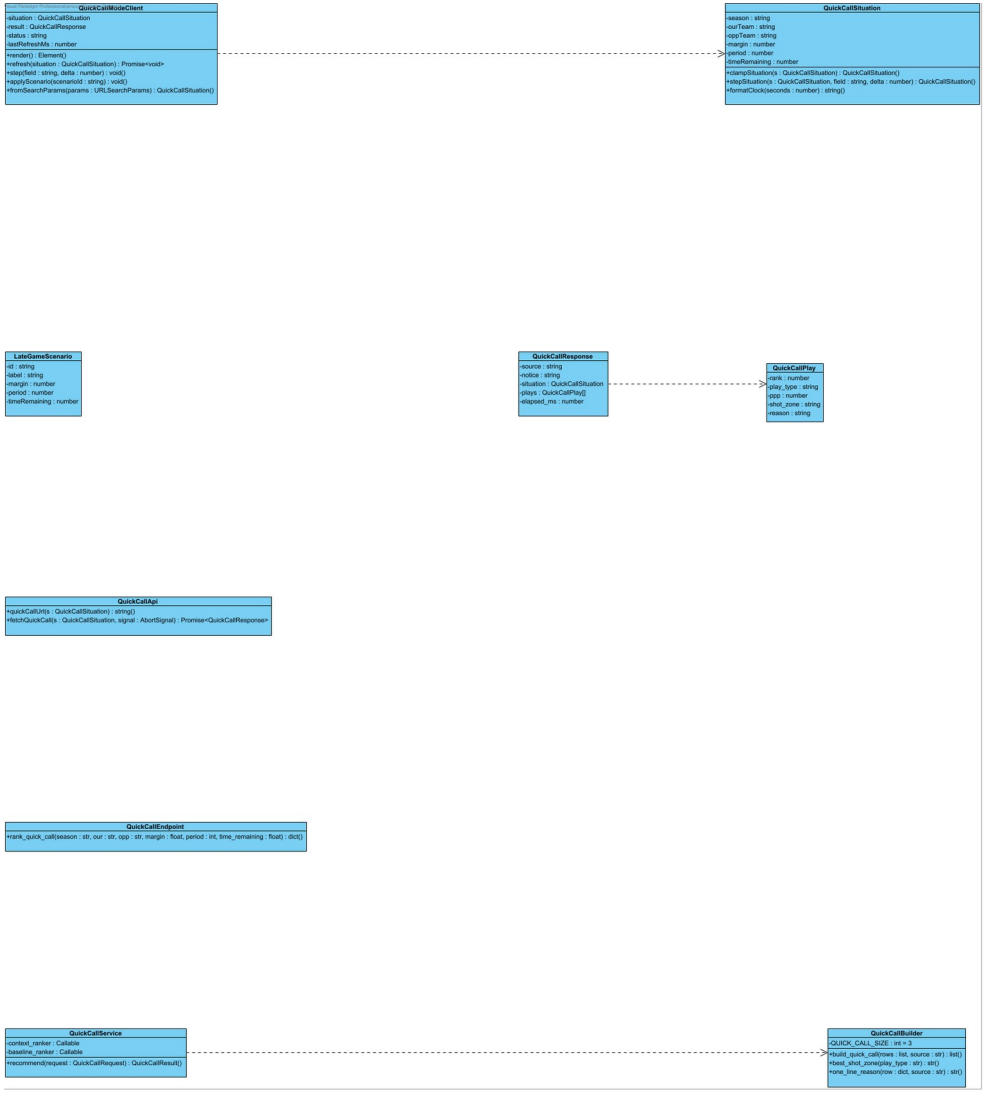
4.12 Timeout Quick-Call Mode

During a timeout the coach picks a preset situation and gets the top three plays, a shot zone and a short reason on one screen. The call is cancelled if the inputs change, and a stale response is never shown.

4.12.1 Collaboration Overview - Timeout Quick-Call Mode



4.12.2 Structural Overview - Timeout Quick-Call Mode

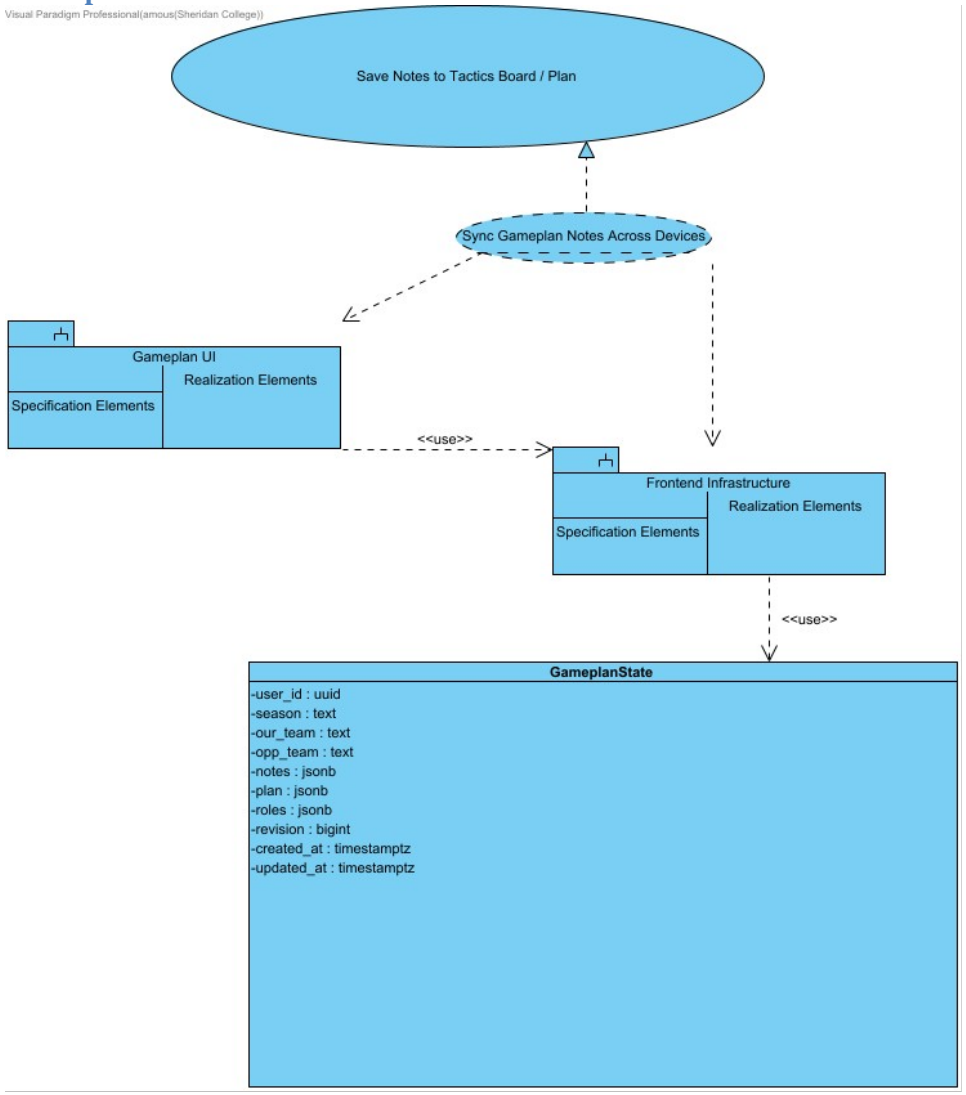


The behavioural design for this interaction is the Quick-Call Refresh and Fallback sequence, shown under Recommendation Services in the Detailed Design section.

4.13 Sync Gameplan Notes Across Devices

The Gameplan board saves the coach's notes and plan to Supabase. If there is no session or Supabase is down, the notes still save on the device and the coach is told. A stale revision is reported as a conflict with the latest copy instead of overwriting another tab's work.

4.13.1 Collaboration Overview - Sync Gameplan Notes Across Devices



The structural design for this interaction is in the Frontend Infrastructure and Gameplan UI subsystem diagrams, and the behavioural design is the Gameplan Sync and Device Fallback sequence shown under Frontend Infrastructure.